eTiQa

UTM UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UTM Johor Bahru

SCSE2123: Software Requirements Engineering

Semester 01, 2025/2026

Software Requirements Specification

Week 13, 3/1/2026

Prepared by: Group 9

Group name: Visionscope

NAME	MATRIC NO
LEE MING DA	A24CS0317
GAN RUI EN	A24CS0249
TAN CHOON DA	A24CS0301
Mohammed Mohsen Mohammed Al-Sakkaf	A23CS4026

Revision Page

a. Overview

This SRS document is organized to provide a complete technical specification for the Etiqa eMoney Payout System. Section 2 establishes the system context, detailing the product perspective, user characteristics, and general constraints to provide a background for the requirements. Section 3 details the specific functional requirements, categorized into three key modules: Claims Processing, Withdrawals & Transfers, and Reinvestment & Compliance. This section also defines the external interface requirements and non-functional attributes such as performance and design constraints. Finally, the Appendices provide supporting documentation, including the originality report and the list of generative AI prompts used during the project's creation

b. Target Audience

The intended audience for this Software Requirements Specification includes:

- Technical Development Team & System Architects: To guide the technical implementation, backend logic, and system integration.
- Project Stakeholders (Etiqa Departments): specifically the Claims, Finance, and Compliance departments, to verify that the business rules and regulatory frameworks (RMIT) are correctly captured.
- Quality Assurance Team: To design test plans and validation criteria based on the specified functional flows.

c. Project Team Members

Member	Assigned Module
Mohammed Mohsen Mohammed Al-Sakkaf	3.2.1 Module User Management & Security
Lee Ming Da	3.2.2 Module Claims Processing
Gan Rui En	3.2.3 Module Withdrawals & Transfers
Tan Choon Da	3.2.3 Module Withdrawals & Transfers

d. Version Control History

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Tan Choon Da Gan Rui En Lee Ming Da Mohammed Mohsen Mohammed Al-Sakkaf	1. Introduction 2. Overall Description	**26 Dec 2025**
2.0	Tan Choon Da Gan Rui En Lee Ming Da Mohammed Mohsen Mohammed Al-Sakkaf	1. Use Case Specification 2. Requirements and Constraints	**2 Jan 2026**

Table of Contents

1. Introduction.....4

1.1 Purpose.....4

1.2 Scope.....4

1.3 Definitions, Acronyms and Abbreviation.....4

1.4 References.....5

1.5 Overview.....5

2. Overall Description.....6

2.1 Product Perspective.....8

2.2 Product Functions.....12

2.3 User Characteristics.....14

2.3.1 The Customer (Primary End-User).....14

2.3.2 The Claims Officer (Internal User).....14

2.3.3 The Compliance Officer (Internal User).....15

2.3.4 System Administrator / IT Operations.....16

2.4 Constraints.....17

2.5 Assumption and Dependencies.....18

2.5.1 Assumptions.....18

2.5.2 Dependencies.....19

3. Specific Requirements.....20

3.1 External Interface Requirements.....24

3.1.1 User Interfaces.....24

3.1.2 Hardware Interfaces.....25

3.1.3 Software Interfaces.....25

3.1.4 Communication Interfaces.....26

3.2 System Features.....27

3.2.1 Module User Management & Security.....27

3.2.1.1 Login to Etiqa+.....27

3.2.1.2 Account Takeover.....28

3.2.1.3 Block Attack (Security Mitigation).....30

3.2.1.4 View Transaction History.....32

3.2.2 Module Claims Processing....34
3.2.2.1 UC004 Request Payout....34
3.2.2.2 UC005 Upload Verification Document....35
3.2.2.3 UC006 Process Incoming Payout....37
3.2.2.4 MUC02 Submit Fraudulent Claim....39
3.2.2.5 SC002 Consolidate Payout Data....41
3.2.3 Module Withdrawals & Transfers....43
3.2.3.1 UC007 Withdraw Funds....43
3.2.3.2 UC009 Generate Transfer Report....46
3.2.3.3 UC014 Update User Balance....48
3.2.3.4 MUC03 Intercept Fund Transfer....51
3.2.3.4 SC003 Manage Verified Bank Accounts....54
3.2.4 Module Reinvestment & Compliance....57
3.2.4.1 UC002 View Account Balance....57
3.2.4.2 UC010 Reinvest Product....60
3.2.4.3 UC015 View Policy Portfolio....64
3.2.4.4 MC004 Bypass Compliance Check....67
3.2.4.5 SC004 Perform Regulatory Check....71
3.3 Performance Requirements....74
3.4 Design Constraints....74
3.5 Software System Attributes....75
3.6 Other Requirements....76
Appendices....76

1. Introduction

This section provides a technical overview of the Software Requirements Specification (SRS) for the Etiqa Digital Payout (eMoney) system. It outlines the project's purpose, scope, and the structural organization of this document.

1.1 Purpose

The primary purpose of this SRS is to provide a complete and unambiguous specification of the functional and non-functional requirements for the Etiqa eMoney Platform. It serves as a formal agreement between project stakeholders (Etiqa Management) and the technical development team. This document is intended to guide software architects during the design phase, developers during implementation, and Quality Assurance (QA) engineers for system validation and testing.

1.2 Scope

The software product being developed is the Etiqa eMoney Platform, a digital wallet feature integrated into the existing Etiqa+ mobile application.

- **What it will do:** The system will automate the disbursement of policy claims, refunds, and rewards directly into a digital wallet. It enables users to withdraw funds to verified bank accounts or reinvest them into other Etiqa insurance products. It includes automated fraud detection and real-time transaction history.
- **What it will NOT do:** It is a closed-loop system; it will not support external retail payments at third-party merchants outside the Etiqa ecosystem.
- **Benefits & Goals:** The system aims to replace the manual 3-to-5-day bank transfer process with instant digital crediting, reducing operational overhead and improving customer liquidity and trust.

1.3 Definitions, Acronyms and Abbreviation

Term / Acronym	Definition
SRS	Software Requirements Specification
BNM	Bank Negara Malaysia (The Central Bank of Malaysia)
RMiT	Risk Management in Technology (Regulatory framework by BNM)
MFA	Multi-Factor Authentication (Security layer for transaction verification)

OTP	One-Time Password
Closed-loop	A payment system where funds are restricted for use within a specific issuer's ecosystem.
DMS	Document Management System
KYC	Know Your Customer (Identity verification process)

1.4 References

1. IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
2. Bank Negara Malaysia (2023). Risk Management in Technology (RMiT) Policy Document.
3. Faculty of Computing, UTM (2025). Case Study: Etiqa Digital Payout (via eMoney).
4. Wiegers, K., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.

1.5 Overview

This document follows the standard SRS structure and is organized as follows:

- **Section 2 (Overall Description):** Provides the context for the system, describing user characteristics (Customers, Claims Officers, Admins), high-level functional diagrams, and design constraints.
- **Section 3 (Specific Requirements):** Contains the core technical details, including detailed Use Case Specifications (3.2.1 to 3.2.4), Data Models (Class Diagrams), and Behavioral Models (State Transition Diagrams).
- **Section 4:** Outlines the Performance Requirements and Software System Attributes.
- **Appendices:** Includes the Turnitin originality report and evidence of tools used.

2. Overall Description

Figure 2.1 shows the Use Case Diagram (UCD) for the Etiqa eMoney System. There is a total of seven actors involved in the system: the **Customer** (Primary Actor), the **Attacker** (Threat Agent), and five supporting system actors (**Payout Service System, Source System, Document Management System, Bank System, and Etiqa+ App**). The system functionalities are categorized into four distinct modules: *Account & Security Management*, *Claims Processing*, *Withdrawal & Transfers*, and *Reinvestment & Compliance*.

In Module 1, the Customer interacts with account security and history features, where the system must defend against account takeover threats. Module 2 handles the inflow of funds, allowing customers to request payouts and upload documents while the system mitigates fraudulent claims. Module 3 manages the outflow of funds, enabling withdrawals to external banks while securing the transfer process against interception. Finally, Module 4 focuses on keeping funds within the ecosystem through reinvestment, ensuring strictly regulated compliance checks are performed.

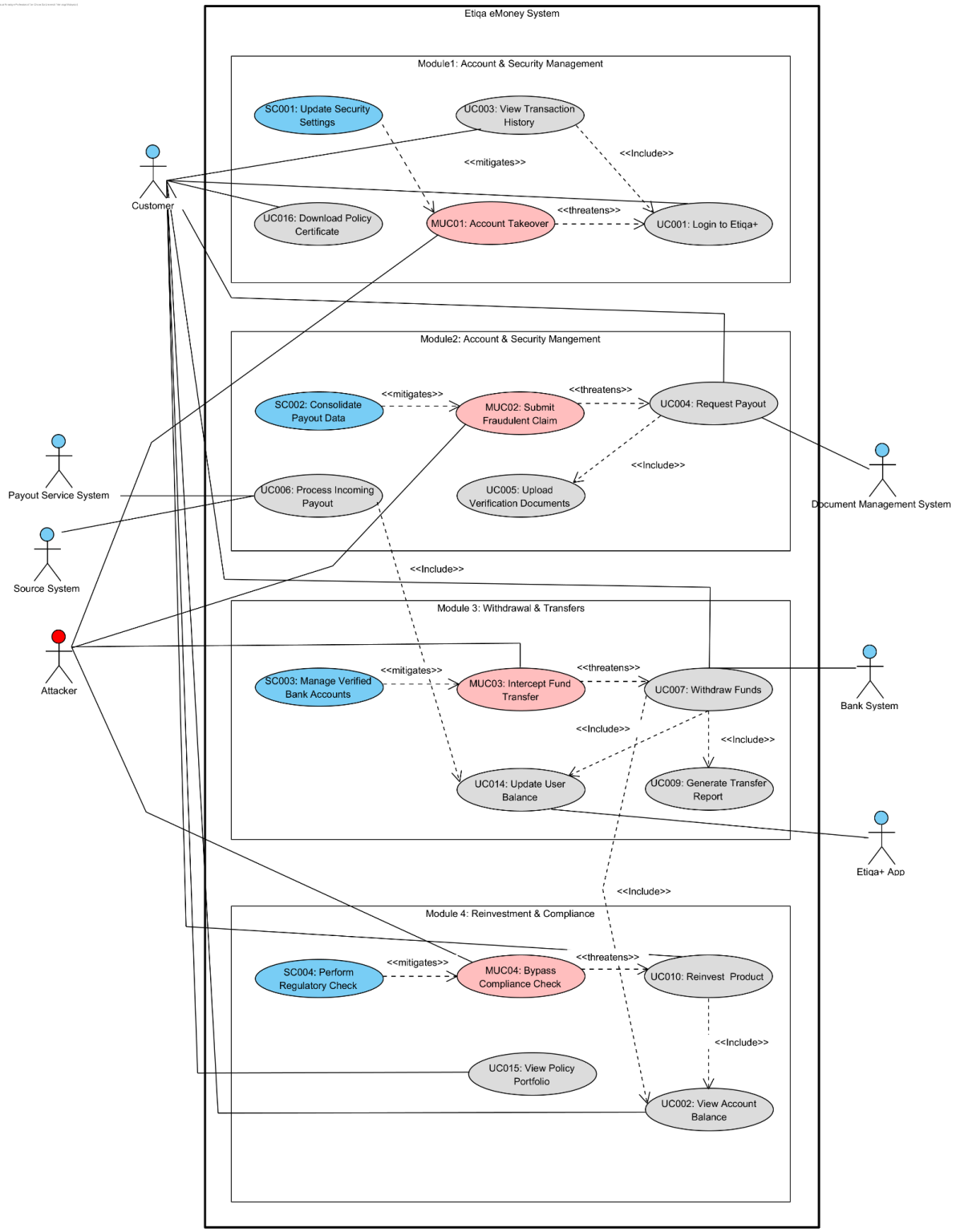


Figure 2.1: Use Case Diagram of <Etiqa eMoney System>

2.1 Product Perspective

The Etiqa eMoney Payout System is a unified, secure and efficient digital payout ecosystem designed to operate as a middleware solution within the Etiqa+ environment. According to Figure 2.1.1, there are eight major sub-goals, with a primary focus on enabling instant payout processing, supporting customer reinvestment, ensuring secure withdrawals and strengthening security compliance.

In order to achieve instant and accurate payout processing, the system will be able to automate the validation of claims by cross-referencing policy limits with the legacy Source of Truth database and crediting funds immediately to the user's eWallet. In terms of secure withdrawals, the system will need to ensure financial safety by strictly allowing transfers only to verified bank accounts and enforcing Multi-Factor Authentication (MFA) for every transaction. The sub-goal for supporting customer reinvestment is to provide a seamless option for users to purchase new products using their available wallet balance, while simultaneously performing real-time regulatory checks (AML/CFT) to prevent illicit activities. Meanwhile, the system will be able to automatically freeze accounts if suspicious patterns are detected. Lastly, the system shall provide real-time transparency and fund visibility by generating immutable audit logs and transaction history reports, so that the stakeholders can audit operational efficiency and ensure compliance with BNM RMIT guidelines.

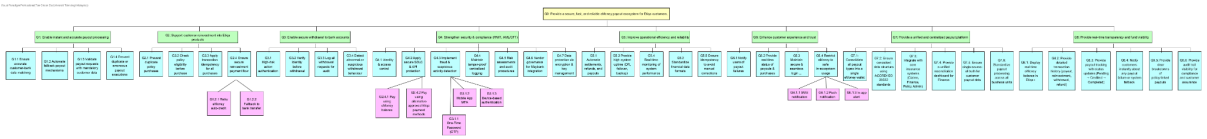


Figure 2.1.1: Goal Model of <Proposed Etiqa eMoney Payout and Reinvestment System>

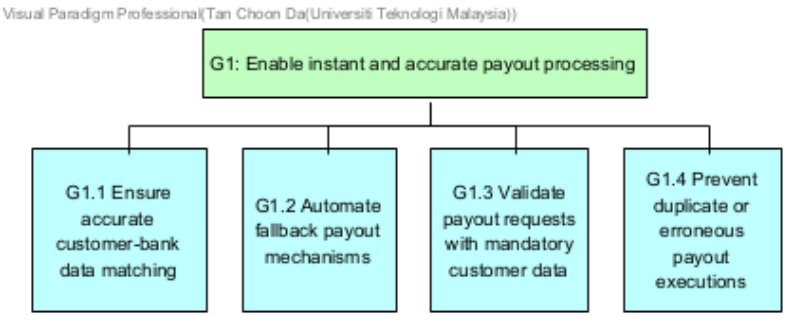


Figure 2.1.2: G1: Enable instant and accurate payout processing

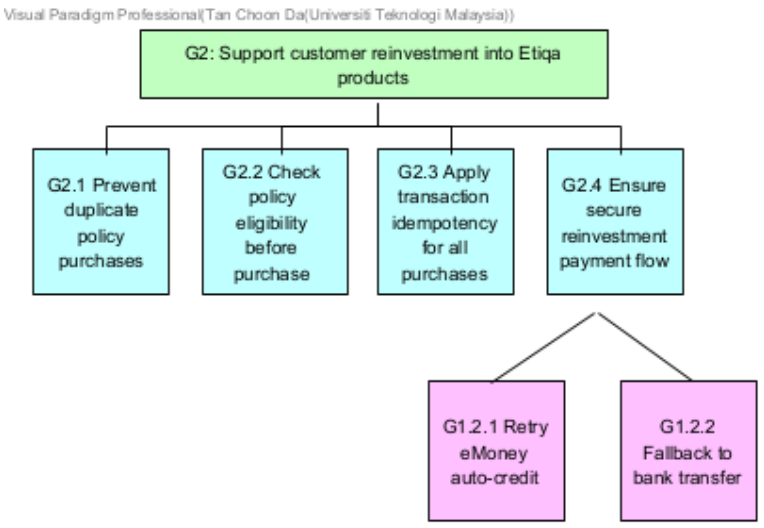


Figure 2.1.3: G2: Support customer reinvestment into Etiqa products

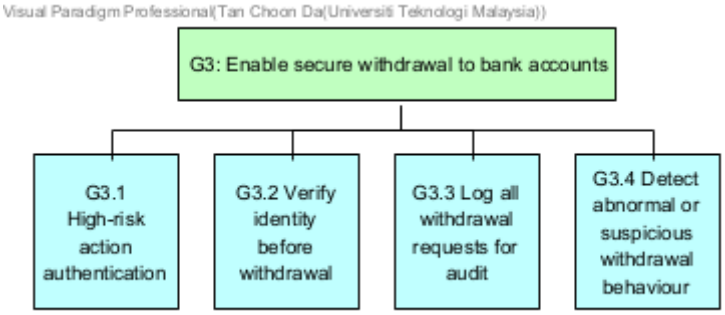


Figure 2.1.4: G3: Enable secure withdrawal to bank account

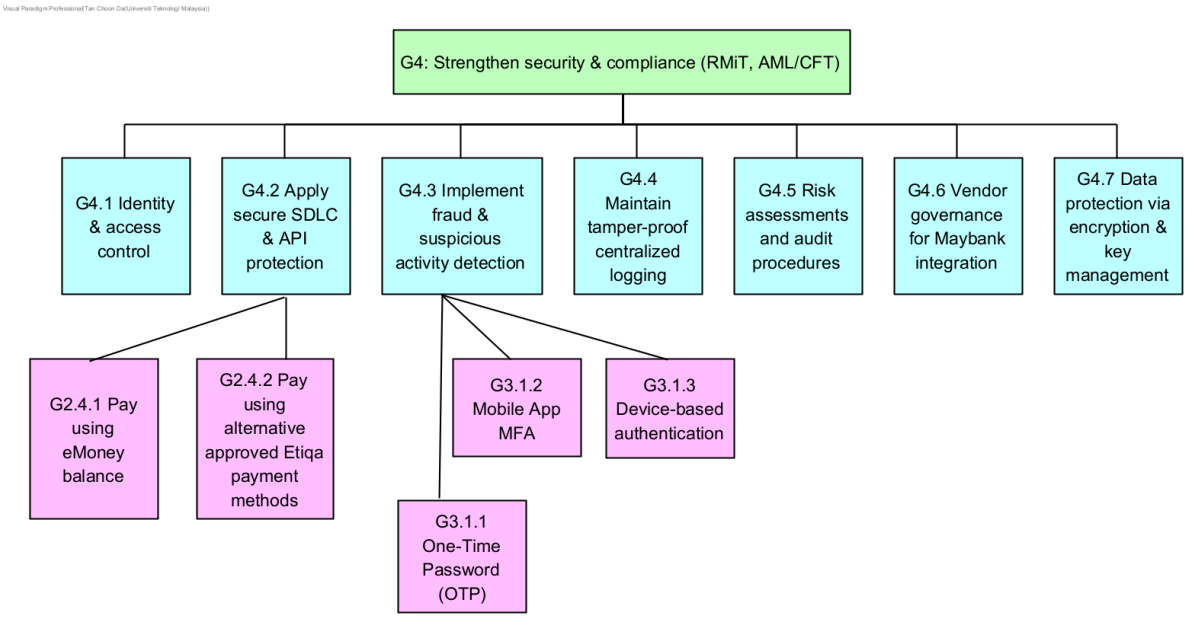


Figure 2.1.5: G4: Strengthen security & compliance (RMiT, AML/CFT)

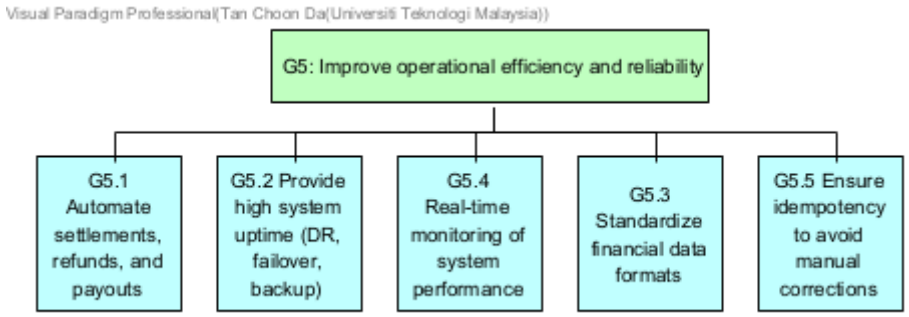


Figure 2.1.6: G5: Improve operational efficiency and reliability

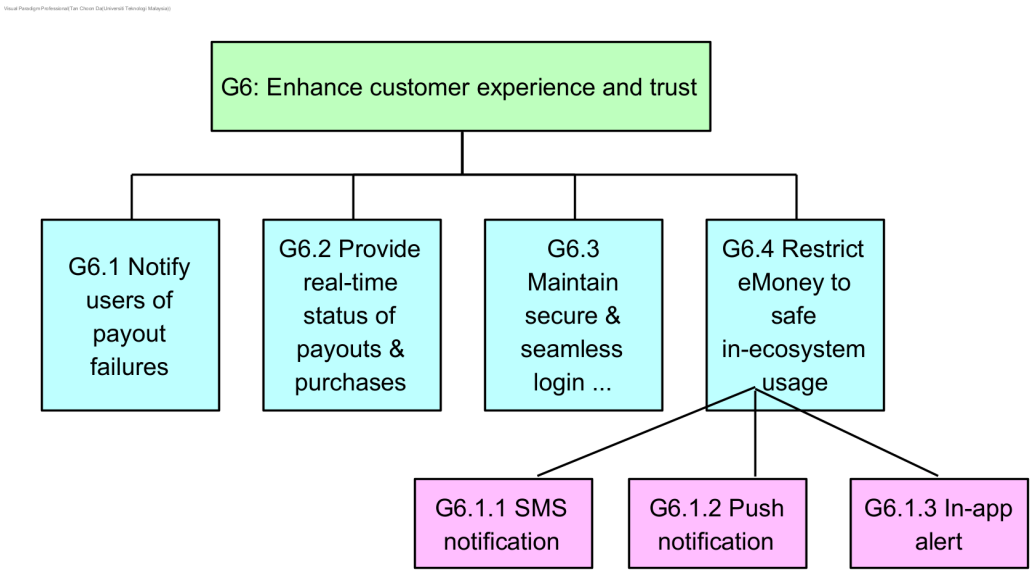


Figure 2.1.7: G6: Enhance customer experience and trust

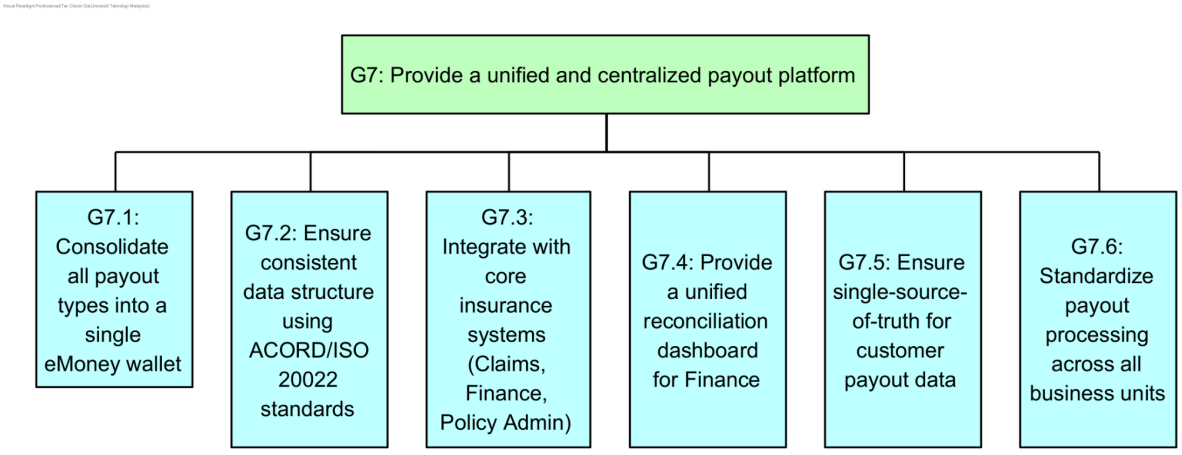


Figure 2.1.8: G7: Provide a unified and centralized payout platform

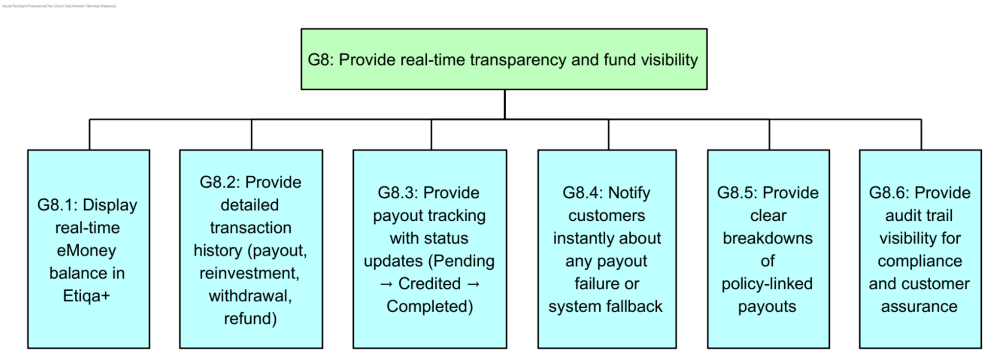


Figure 2.1.9: G8: Provide real-time transparency and fund visibility

The vision of the proposed Etiqa eMoney Payout System is to establish a unified, secure, and efficient digital payout ecosystem that enhances customer satisfaction while complying with regulatory and security requirements. It also aims to smoothen payout processing, support reinvestment, enhance security controls, and improve operational efficiency across Etiqa's digital platforms.

The Goal Diagram (Figure 2.1) visually represents this system vision by decomposing the top-level goal (G0: Provide a secure, fast, and reliable eMoney payout ecosystem for Etiqa customers) is expanded into eight major goal areas:

- G1: Enable instant and accurate payout processing
- G2: Support customer reinvestment into Etiqa products
- G3: Enable secure withdrawal to bank accounts
- G4: Strengthen security & compliance (RMiT, AML/CFT)
- G5: Improve operational efficiency and reliability
- G6: Enhance customer experience and trust
- G7: Provide a unified and centralized payout platform
- G8: Provide real-time transparency and fund visibility

These high-level goals are further broken down into detailed operational goals, which represent the specific behaviors, rules, and capabilities that the system must provide. The hierarchical decomposition of the root goal into structured sub-goals using both AND and OR decomposition provides a clear, high-level understanding of how the proposed system will achieve its overall objective. The diagram captures how system vision is supported by both functional and non-functional requirements.

2.2 Product Functions

Module 1: Account & Security Management

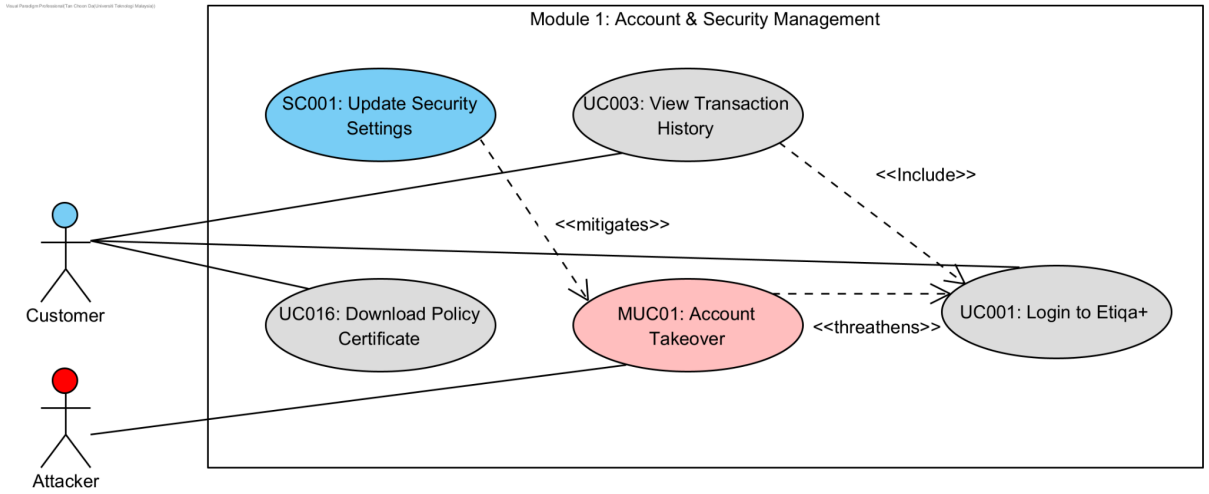


Figure 2.2.1: Use Case Diagram for Module 1

- **Focus:** Managing user access, historical visibility, and document security.
- **Functional Features:**
    - **Login to Etiqa+ (UC001):** Secure user authentication entry point.
    - **View Transaction History (UC003):** Allows users to audit their past inflows and outflows.
    - **Download Policy Certificate (UC016):** Retrieval of official digital policy documents.
- **Security & Mitigation:**
    - **Update Security Settings (SC001):** Allows users to change passwords or MFA settings. Mitigates MUC01 (Account Takeover).

Module 2: Claims Processing

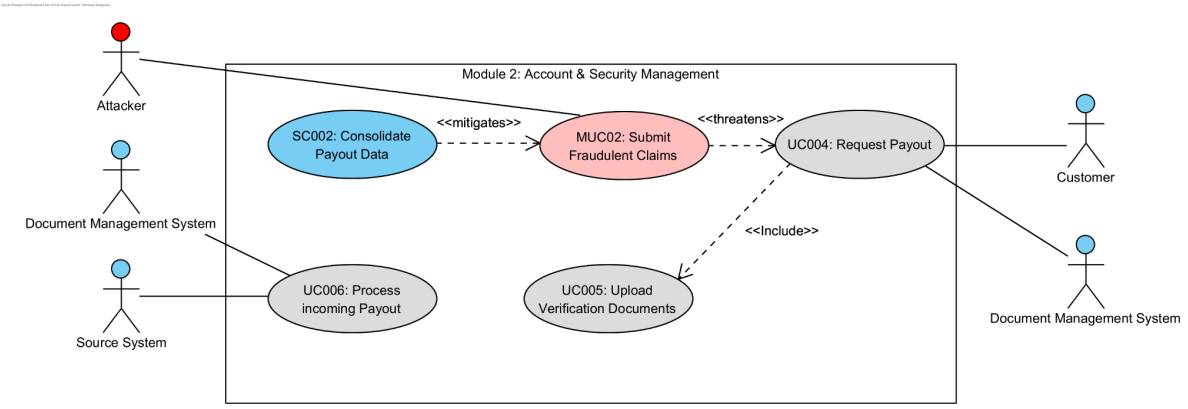


Figure 2.2.2: Use Case Diagram for Module 2

- **Focus:** The incoming money flow which requests and processes payouts.
- **Functional Features:**
    - **Request Payout (UC004):** The primary initiation of a claim by the user.
    - **Upload Verification Documents (UC005):** Handling evidence submission (Included in UC004).
    - **Process Incoming Payout (UC006):** Backend logic to validate and accept funds into the system.
- **Security & Mitigation:**
    - **Consolidate Payout Data (SC002):** Verifies consistency across data sources. Mitigates MUC02 (Submit Fraudulent Claim).

Module 3: Withdrawals & Transfers

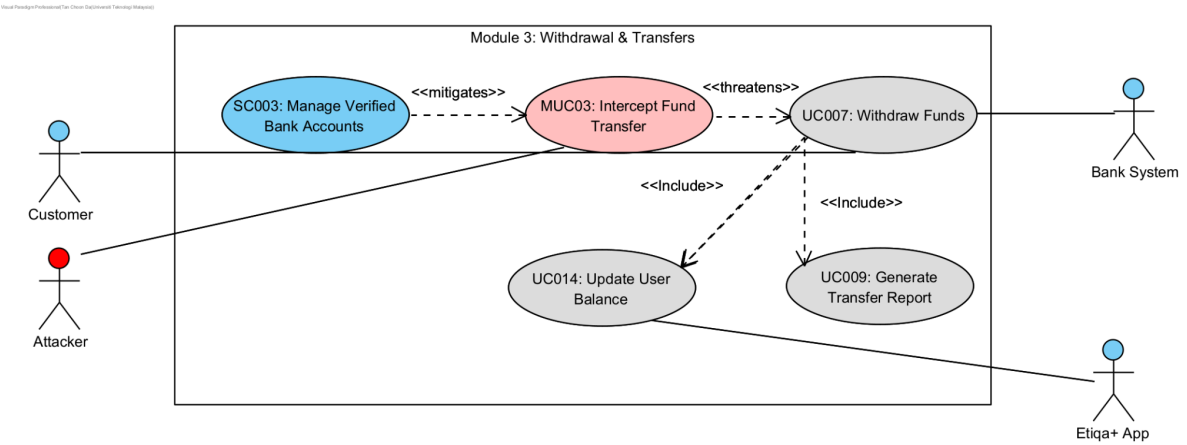


Figure 2.2.3: Use Case Diagram for Module 3

- **Focus:** The outgoing money flow which moves money from the eWallet to external banks.
- **Functional Features:**
    - **Withdraw Funds (UC007):** Initiating the transfer of funds to a bank account.
    - **Generate Transfer Report (UC009):** Creating receipts/logs for the transaction (Included in UC007).
    - **Update User Balance (UC014):** Deducting the ledger balance post-withdrawal.
- **Security & Mitigation:**
    - **Manage Verified Bank Accounts (SC003):** Restricting transfers to pre-verified accounts only. Mitigates MUC03 (Intercept Fund Transfer).

Module 4: Reinvestment & Compliance

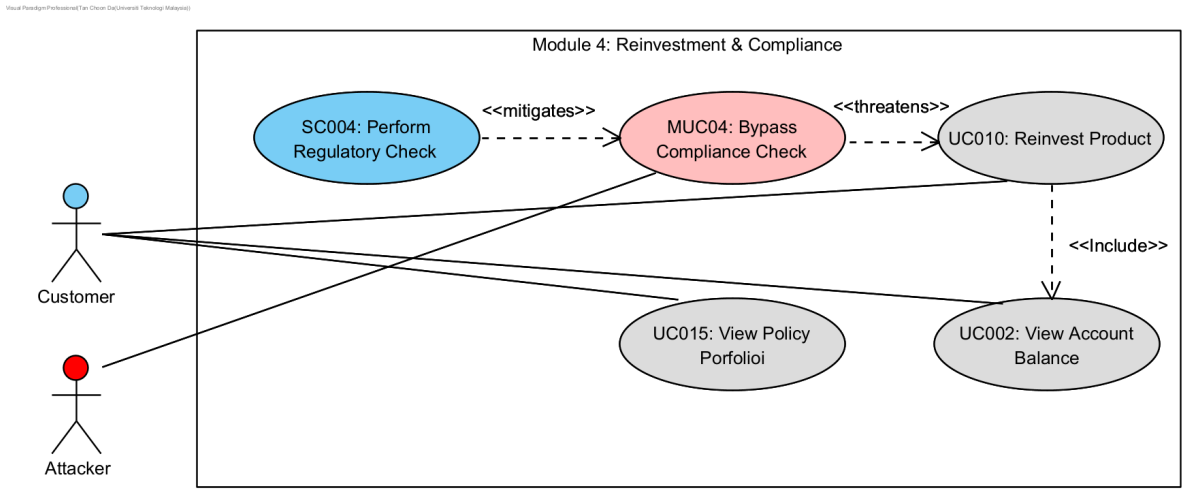


Figure 2.2.4: Use Case Diagram for Module 4

- **Focus:** Keeping money within the ecosystem and ensuring regulatory adherence.
- **Functional Features:**
    - **Reinvest Product (UC010):** Using eWallet balance to purchase new insurance products.
    - **View Policy Portfolio (UC015):** Displaying current active holdings.
    - **View Account Balance (UC002):** Checking available funds (Prerequisite for reinvestment).
- **Security & Mitigation:**
    - **Perform Regulatory Check (SC004):** Automated screening against watchlists. Mitigates MUC04 (Bypass Compliance Check).

2.3 User Characteristics

2.3.1 The Customer (Primary End-User)

The primary actors interacting with the system are Etiqa policyholders who use the mobile application to request payouts, withdraw funds, and reinvest products.

- **Educational Level:** Highly variable. The user base ranges from secondary school graduates to post-graduates. The system cannot assume a high level of financial literacy or understanding of complex insurance terminology.
- **Experience:** Users are generally familiar with standard mobile applications such as social media, basic banking. However, they may be unfamiliar with the specific "eWallet" concept within an insurance context, as they are accustomed to the older manual bank transfer method.
- **Technical Expertise:** Low to Moderate. Most users can navigate touch interfaces but may struggle with troubleshooting technical errors or understanding file format restrictions during document uploads.

- **Impact on Requirements:**
    - Since users experience anxiety regarding "funds in transit", the system must provide real-time visibility of balances (UC002) and immediate feedback (success/fail messages) to prevent users from contacting support unnecessarily.
    - Due to the variance in technical skill, the Document Upload process (UC005) must include automated validation for file types and sizes to prevent user frustration before submission .
    - Security measures like MFA and OTP (SC001) must be implemented seamlessly to protect users who may be prone to phishing or using weak passwords.

2.3.2 The Claims Officer (Internal User)

These are back-office operational staff responsible for managing flagged transactions and reviewing payout requests that fail automated verification.

- **Educational Level:** Diploma or Bachelor's degree holders in Finance, Business Administration, or Insurance.
- **Experience:** Highly experienced in Etiqa's internal policy frameworks and legacy systems. They are accustomed to cross-referencing claims against the "Source of Truth" legacy databases.
- **Technical Expertise:** Moderate. They are proficient in using desktop-based enterprise software and interpreting data logs but are not software developers.
- **Impact on Requirements:**
    - Because these users must verify data discrepancies manually when automation fails, the system must generate detailed Data Mismatch Flags and logs (SC002) rather than generic error codes.
    - The system must provide clear audit trails for every transaction state change For example, from "Pending" to "Approved" or "Rejected" to support their daily operational workflow .

2.3.3 The Compliance Officer (Internal User)

These are specialized staff members responsible for ensuring the system adheres to Bank Negara Malaysia (BNM) regulations and RMIT policies.

- **Educational Level:** Bachelor's or Master's degree in Law, Finance, or Risk Management.
- **Experience:** Expert level knowledge regarding Anti-Money Laundering (AML) and Counter-Financing of Terrorism (CFT) laws. They rely on the system to detect complex patterns like "structuring" (splitting large transactions into small ones) .
- **Technical Expertise:** High functional knowledge of compliance tools but low patience for false negatives (missed fraud). They require absolute data accuracy.
- **Impact on Requirements:**
    - The critical need to enforce regulatory reporting limits drives the requirement for cumulative transaction tracking (SC004) rather than just single-transaction checks.

    - The requirement for immediate account freezing (MUC03, MUC04) is derived from their need to neutralize threats instantly upon detection of a watchlist match or suspicious behavior.

2.3.4 System Administrator / IT Operations

These are technical staff responsible for the maintenance, availability, and security of the Etiqa eMoney infrastructure.

- **Educational Level:** Bachelor's degree in Computer Science, Information Technology, or Network Engineering.
- **Experience:** High experience in managing Microservices Architecture, API gateways, and SQL/NoSQL databases.
- **Technical Expertise:** Expert. They must be capable of interpreting the Security Logs (SecurityLog entity) and investigating "Security Incidents" generated by the system during attacks.
- **Impact on Requirements:**
    - The requirement for immutable audit logs (Section 3.6.1 of SRS) is specifically for this user group to trace system failures or security breaches.
    - The Performance Requirements (Section 3.3) regarding throughput (1,000 concurrent users) and uptime (99.9%) are operational constraints managed by this group.

2.4 Constraints

1) Regulatory policies: The system must strictly comply with Bank Negara Malaysia's Risk Management in Technology (RMIT) framework and Anti-Money Laundering (AML) laws, requiring automated account freezing upon detection of suspicious activity .

2) Hardware limitations: The client application is limited to the processing capabilities of standard smartphones, requiring complex logic to be offloaded to the backend, and strictly demands continuous network connectivity for real-time ledger updates .

3) Interfaces to other applications: The system is constrained by the legacy Source System's data formats for policy validation and the external Bank System's API processing speeds, requiring the handling of asynchronous "Pending" states for withdrawals .

4) Parallel operation: To prevent race conditions, the system must enforce strict concurrency controls by locking the user's wallet record during balance updates, effectively preventing simultaneous financial transactions for a single user ID.

5) Audit functions: The system must implement immutable Compliance and Security logs, strictly prohibiting the modification or deletion of any historical transaction records or linked evidence documents.

6) Control functions: Automated control logic, such as daily transaction limits and reinvestment reporting thresholds, must be hard-coded to detect structuring attempts and protected from standard user modification .

7) Reliability requirements: The system must ensure data consistency through ACID properties, automatically performing rollbacks on local ledger updates if the corresponding external bank transaction fails.

8) Criticality of the application: As a financial system replacing manual payouts, the architecture requires mandatory redundancy and fail-over mechanisms to prevent direct financial loss or data corruption during system outages.

9) Safety and security considerations: The system is restricted from storing files without a prior malware scan and must enforce access control by validating device fingerprints to trigger secondary authentication for unrecognized devices.

2.5 Assumption and Dependencies

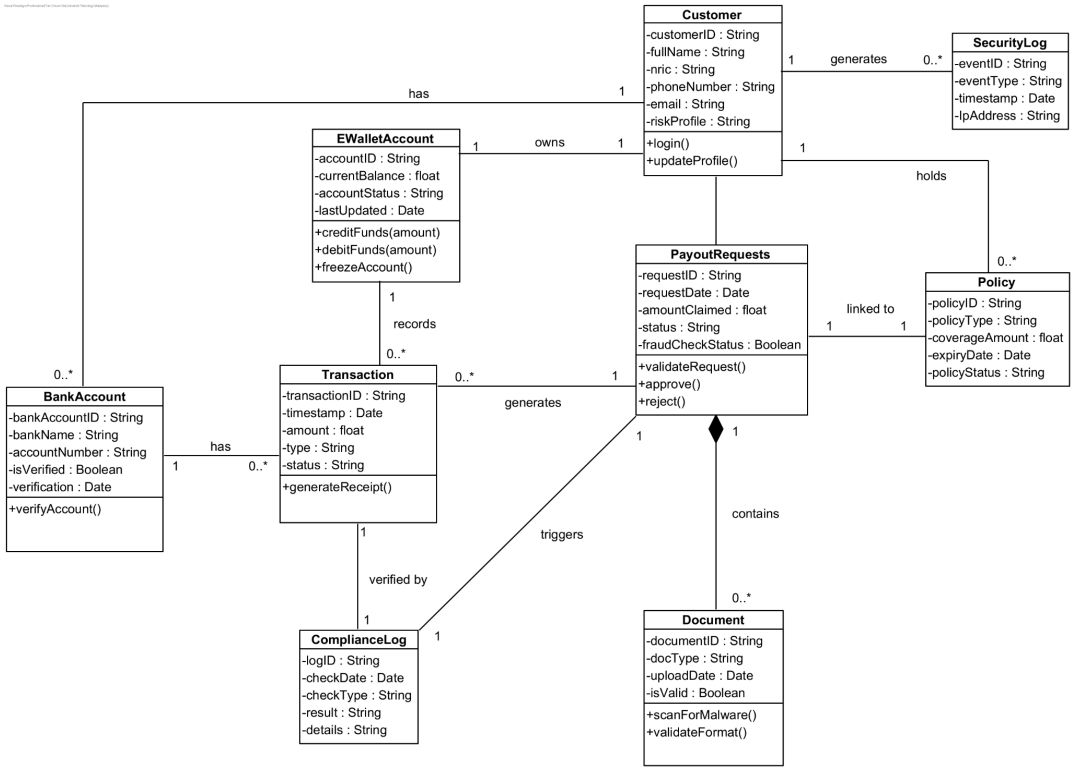
2.5.1 Assumptions

1. **Legacy Data Availability:** It is assumed that the legacy Source System will remain operational and its database schema for retrieving "Original Policy Limit & Status" will not change during the development of this new module, ensuring the Data Reconciliation Service can map data correctly .
2. **User Connectivity:** It is assumed that the Customer will have a stable internet connection (4G/LTE or WiFi) while performing financial transactions, as the system does not support offline mode for real-time fund validation or balance retrieval .
3. **Device Integrity:** It is assumed that the user's mobile device is running a standard, unmodified operating system (non-rooted/jailbroken) to support the security fingerprinting mechanisms required for login protection .
4. **Bank API Stability:** It is assumed that the external banking partners will continue to support the current API standards for Interbank GIRO (IBG) and Instant Transfer without major deprecation during the project lifecycle.

2.5.2 Dependencies

1. **Core Ledger System:** The software is fundamentally dependent on the existing Core Ledger to serve as the "Source of Truth" for user funds; any downtime or latency in the Core Ledger will directly impact the availability of the eMoney features.
2. **External Verification Services:** The "Manage Verified Bank Accounts" feature (SC003) depends on external banking APIs to perform the Know Your Customer (KYC) identity checks; failure of this external service will block users from adding new withdrawal destinations .
3. **Notification Gateways:** The delivery of One-Time Passwords (OTP) for security challenges and transaction alerts depends on third-party SMS and Push Notification providers; delays in this service will prevent legitimate users from authenticating or receiving balance updates.
4. **Regulatory Frameworks:** The logic for the "Perform Regulatory Check" (SC004) is strictly dependent on the current Risk Management in Technology (RMIT) and AML/CFT guidelines provided by Bank Negara Malaysia; any change in these laws will require an immediate update to the system's compliance algorithms .

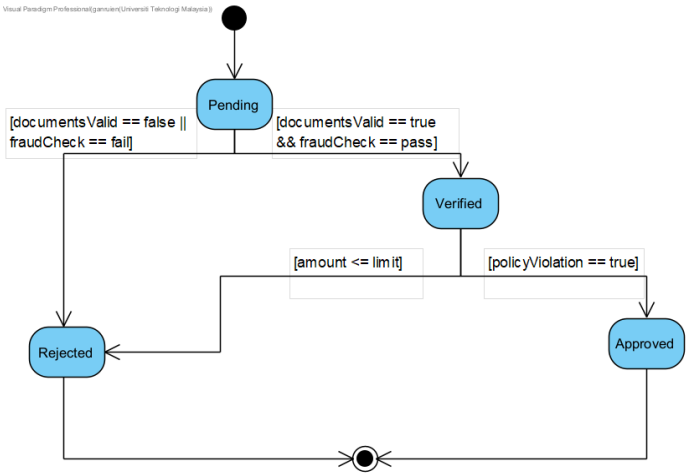
3. Specific Requirements



DM001 - Domain Model of Etiqa Digital Payout

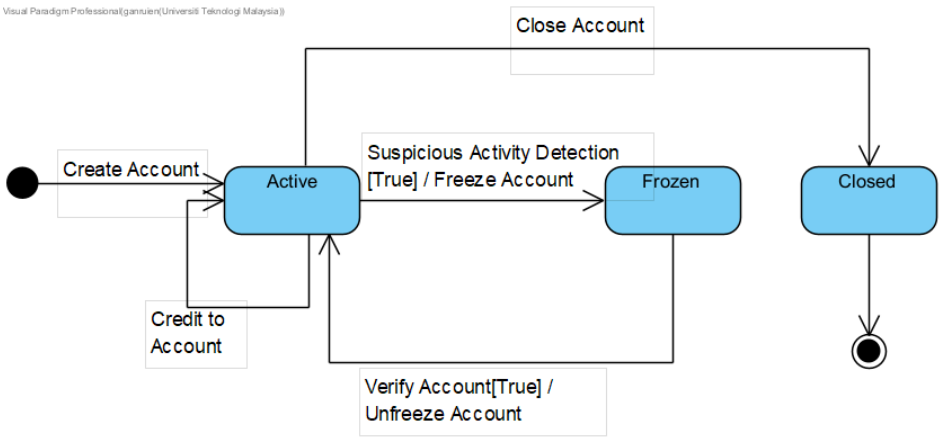
The UML Domain Class Diagram (DCD) illustrates the static view of the Etiqa eMoney System by modeling the underlying data structures, attributes and the relationships between them. This diagram identifies the core business entities required to support the functional use cases such as customer, eWalletAccount, Payout Request, Transaction, ComplianceLog and SecurityLog. This diagram captures the essential data requirements for the system, mapping directly to the business rules defined in the URD.

The Domain Class Diagram illustrates the static data structure of the Etiqa eMoney System, centering on the Customer entity which maintains a strictly one-to-one relationship with an EWalletAccount to ensure a unified ledger for managing the currentBalance. Customers can initiate PayoutRequests, which are fundamentally linked to a single Policy as the source of truth and are composed of multiple Documents that serve as mandatory evidence. The successful requests will generate Transaction that are verified against ComplianceLogs to ensure regulatory adherence while the system concurrently tracks the user behavior through SecurityLogs to monitor for potential threats or anomalies.



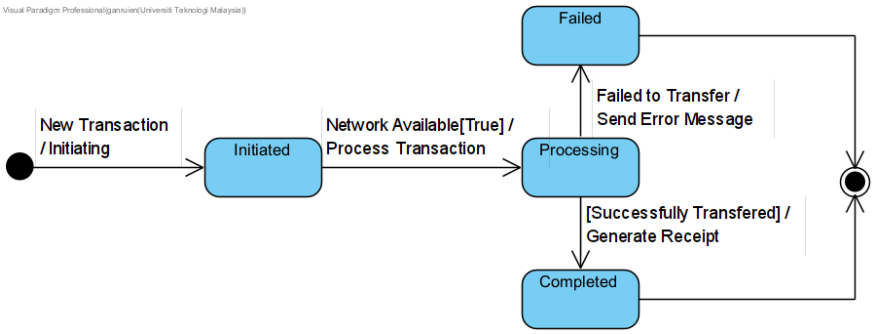
STD001 - State Transition Diagram of Payout Request

The PayoutRequest entity manages the lifecycle of a customer's claim. It begins in a Pending state upon creation. The system immediately triggers validation processes. If the documents and fraud checks pass, it transitions to Verified. If any check fails, it moves directly to Rejected. From the Verified state, the request awaits a final decision of successful authorization transitions it to Approved, triggering the financial transaction, while a policy violation or manual denial moves it to Rejected.



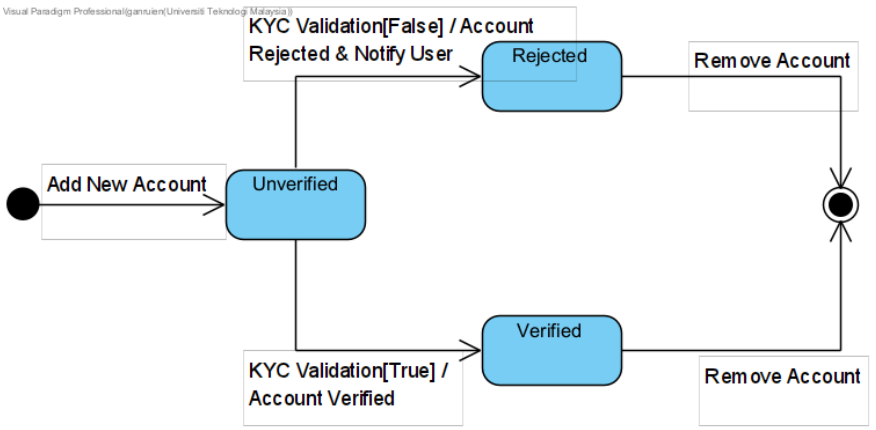
STD002 - State Transition Diagram of e-Wallet Account

The e-Wallet Account entity represents the user's operational fund status. It starts in an Active state that allows standard deposits and withdrawals. If the system detects suspicious behavior, the account transitions to Frozen, temporarily disabling outgoing funds until a verification process restores it to Active. Finally, if a user decides to leave and the balance is zero, the account transitions to Closed, archiving the data and preventing further use.



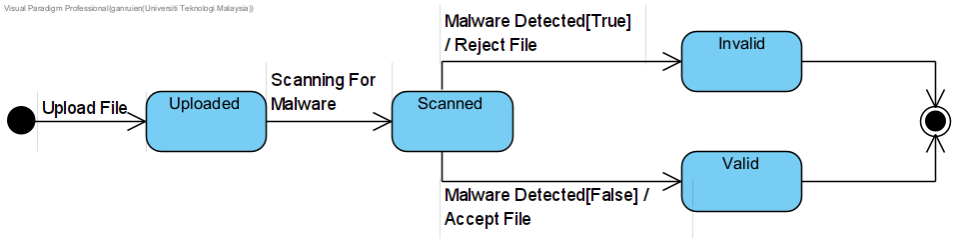
STD003 - State Transition Diagram of Transaction

The Transaction entity tracks the specific movement of funds. It initializes in an Initiated state and quickly moves to Processing as the system communicates with the external banking network. This state represents a critical waiting period. Depending on the response from the bank, the transaction definitively transitions to either Completed and generate a receipt or Failed when error occurs to ensure that the financial status is always atomic and clear.



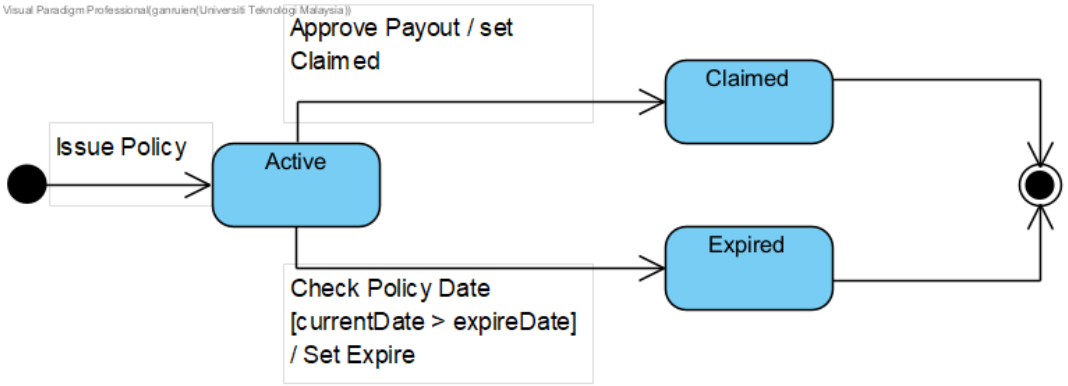
STD004 - State Transition Diagram of Bank Account

The Bank Account entity handles the verification of external withdrawal destinations. A new account starts as “Unverified”. It must typically undergo an identity check which is KYC and bank validation. A match transitions the state to Verified to enable withdrawals meanwhile a mismatch results in a Rejected state. Users can also delete an account at any time, which terminates the entity's lifecycle regardless of its current status.



STD005- State Transition Diagram of Document

The Document entity tracks the file validation process for claims. Upon upload, a file enters the Uploaded state. It automatically transitions to Scanned as the system performs anti-malware checks. Based on the scan results and format compatibility, the document ultimately settles as either Valid or Invalid to ensure only safe data enters the system's processing workflow.



STD006 - State Transition Diagram of Policy

The Policy entity represents the insurance product's status. It begins as Active when issued to the customer. The system monitors the policy against two conditions which are time and utilization. If the current date passes the policy's expiry date without a claim, it transitions to Expired. Alternatively, if a valid claim payout equals the total coverage amount, it transitions to Claimed.

3.1 External Interface Requirements

3.1.1 User Interfaces

The system shall provide a responsive mobile application interface for Customers and a web-based dashboard for Claims Officers/Admins.

Mobile Application (Customer):

Dashboard: Display the current eWallet balance, quick action buttons such as withdraw and reinvest button, and the recent transaction history.

Claims Section: Provide a form to customers to select the policies, input claims amounts, and an upload interface for verification documents.

Notification Area: Display push notifications for payout approvals and balance updates.

Portfolio View: Provide a consolidated list of active policies and the corresponding values.



Figure for the User Interface of Mobile View

Web Portal (Admin/Backend):

Login Screen: Standard secure login for authorized staff.

Transaction Monitor: Display a log of all incoming payouts and their approval status.

Exception Handling: Display the flagged transaction to staff for further solution.

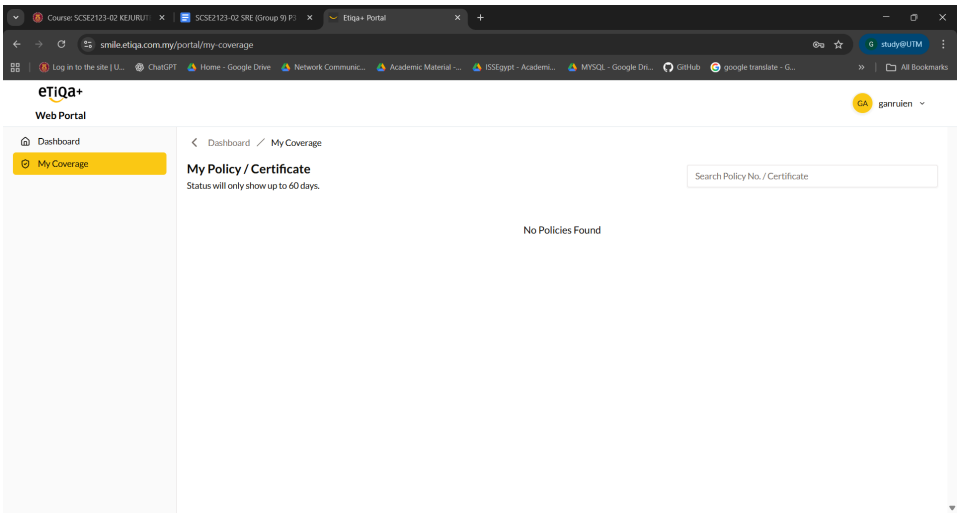


Figure for the User Interface of PC User View

3.1.2 Hardware Interfaces

Any type of devices such as PC, tablet and mobile will act as an interface to utilize Etiqa+ App. The minimum recommended hardware requirements are described below:

Browser	Processor	RAM	OS	HDD/SDD	Disk Space
All Browsers can be used.	Intel Core i3 (sixth generation or newer) or equivalent	4GB	Windows/Mac OS/Linux	200MB	150MB

3.1.3 Software Interfaces

Name	Specification	Version	Source
BootStrap	Free and open source software used for efficient web development using HTML and CSS based design. Also used to create intuitive user interfaces.	V5.3.8	Open Source
MySQL DBMS	It makes it easy for the developers to manage and create databases in the form of tables which are formatted and it has an open source user interface.	V8.4.7	Open Source
Visual Studio Code	Used to ease web developers in editing the code. Allows the developers to create and manage codes written in different programming languages.	Version 1.97	Open Source

3.1.4 Communication Interfaces

The system shall utilize standard communication protocols to ensure secure data transmission.

Client-Server Communication:

All communication between the Etiqa+ App and the Backend Server shall be encrypted using HTTPS (TLS 1.3) to prevent "Man-in-the-Middle" attacks(MUC003).

Push Notifications:

The system shall utilize standard push notification protocols to alert users of balance updates (UC014).

Data Format:

API data exchange shall use JSON format for lightweight and fast data parsing.

3.2 System Features

3.2.1 Module 1: Account & Security Management

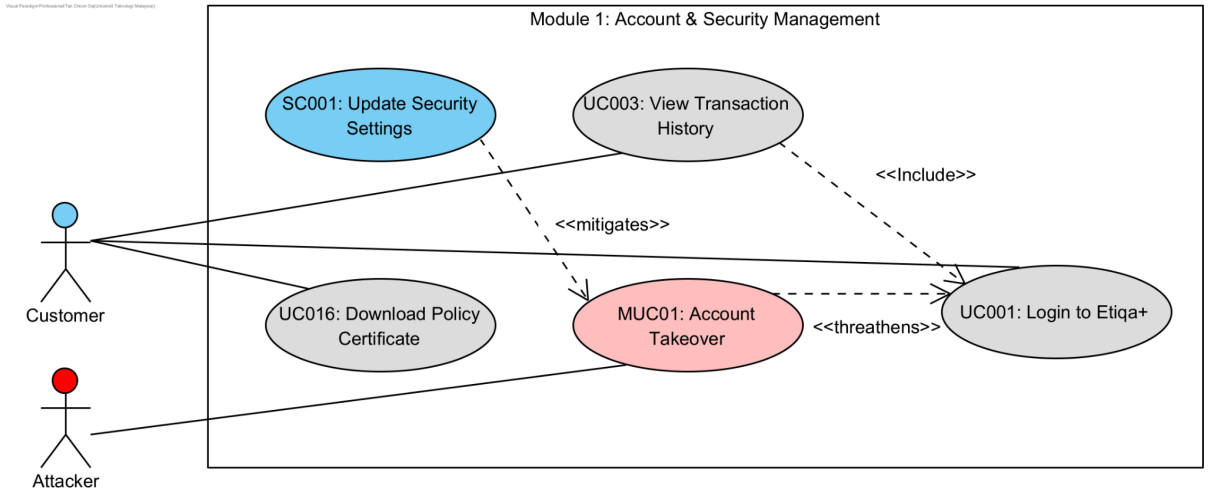


Figure 3.2.1: Use Case Diagram for Module 1

3.2.1.1 UC001 Login to Etiqa+

History Log:	1.0 (22/12/2025)	Initial Draft	
	1.1 (01/01/2026)	1.0 - Initial Draft	
Version:	1.0		
Use Case ID:	UC001		
Use Case Name:	Login to Etiqa+		
Created By:	Mohammed Alsakkaf	**Last Updated By:**	Mohammed Alsakkaf
Date Created:	01 January 2026	**Last Revision Date:**	01 January 2026
Actors:	Customer, Etiqa+ System		
Description:	Allows the user to securely access the eMoney platform using their credentials.		
Pre-conditions:	1. The user has a registered account.		

	2. Etiqa+ App is installed and opened.		
Normal Flow:	1. User enters Username and Password. 2. System validates input format. 3. The system authenticates users against the database. 4. System resets failed login counters. 5. System retrieves user session. 6. The system displays a Home Dashboard.		
Alternative Flow:	A1: Invalid Format (At Step 2) 1. If the format is invalid, the system displays an "Invalid Format" error. 2. Users are prompted to re-enter credentials. 3. Use Case returns to Step 1 (Normal Flow).		
Exception:	E1: Authentication Failed (At Step 3) 1. System increments "Failed Attempts" counter. 2. If Max Attempts (3) reached: System locks account temporarily and sends Security Alert (Email/SMS). 3. System locks account temporarily and use case terminates.		
Post-conditions:	User is successfully authenticated and granted access to the dashboard. A valid session token is generated for the user.		
Related Requirement:	**ID**	**Requirement**	**Priority**
	FR-UC001-01	The system **SHALL** authenticate users using multi-factor credentials	**High**

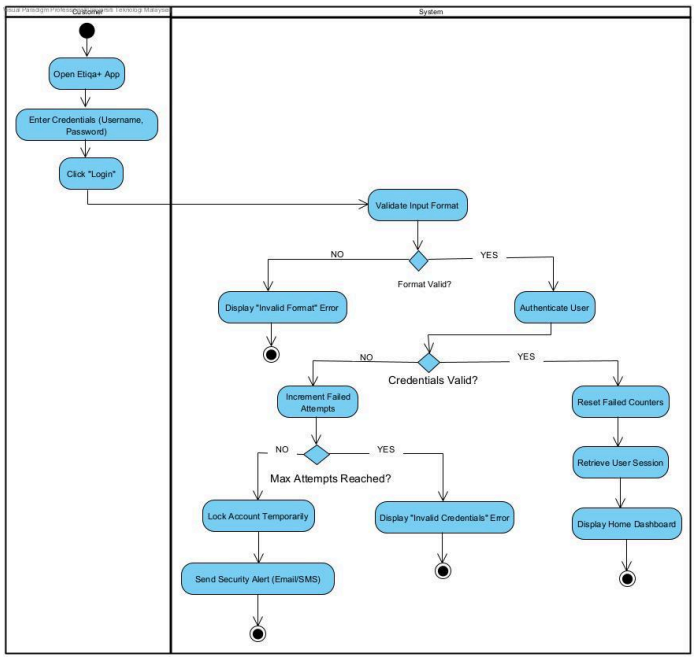


Figure 3.2.1.1: Activity Diagram for UC001 Login to Etiqa+

For UC001, this diagram details the primary authentication process for accessing the Etiqa+ eMoney platform. The flow begins when the user inputs their registered credentials. The system performs an internal validation of the data format before authenticating against the central database. Upon success, the failed attempt counter is reset, and the user is granted a secure session token, ensuring that only authorized individuals can access sensitive financial features.

3.2.1.2 UC003 View Transaction History

History Log:	1.0 (22/12/2025)	Initial Draft	
	1.1 (01/01/2026)	Updated …..	
Version:	1.0		
Use Case ID:	UC003		
Use Case Name:	View Transaction History		
Created By:	Mohammed Alsakkaf	**Last Updated By:**	Mohammed Alsakkaf
Date Created:	01 January 2026	**Last Revision Date:**	01 January 2026
Actors:	Customer, Core Ledger System		
Description:	Allows the user to view, filter, and drill down into their payout and reinvestment history.		
Pre-conditions:	1.User is successfully logged into the Etiqa+ app. 2. User has at least one previous transaction (payout or reinvestment) recorded in the Core Ledger.		
Normal Flow:	1. User selects "Transaction History" from dashboard. 2. User selects Date Range/Category filter. 3. System validates filter parameters. 4. System requests records from Core Ledger System. 5. System sorts records by Date (Descending). 6. System displays list (Date, Type, Amount). 7. User clicks on a specific transaction item. 8. System displays Full Details (ID, Status, Remarks).		
Alternative Flow:	A1: Use Default (At Step 2) 1. User selects "No Filters".		

	2. System applies default 'Last 30 Days' filter and proceeds to Step 3 of Normal Flow.		
Exception:	E1: No Records (At Step 5) 1. If the list is empty: System displays "No Transactions Found". 2. Use case ends.		
Post-conditions:	Users have successfully viewed their financial transaction logs.		
Related Requirement:	**ID**	**Requirement**	**Priority**
	FR-UC003-01	The System **SHALL** maintain a minimum of 12 months of transaction history for user retrieval.	**Basic**
	FR-UC003-02	The System **SHALL** allow the user to filter history by category (Payouts, Reinvestments, Withdrawals).	**Basic**

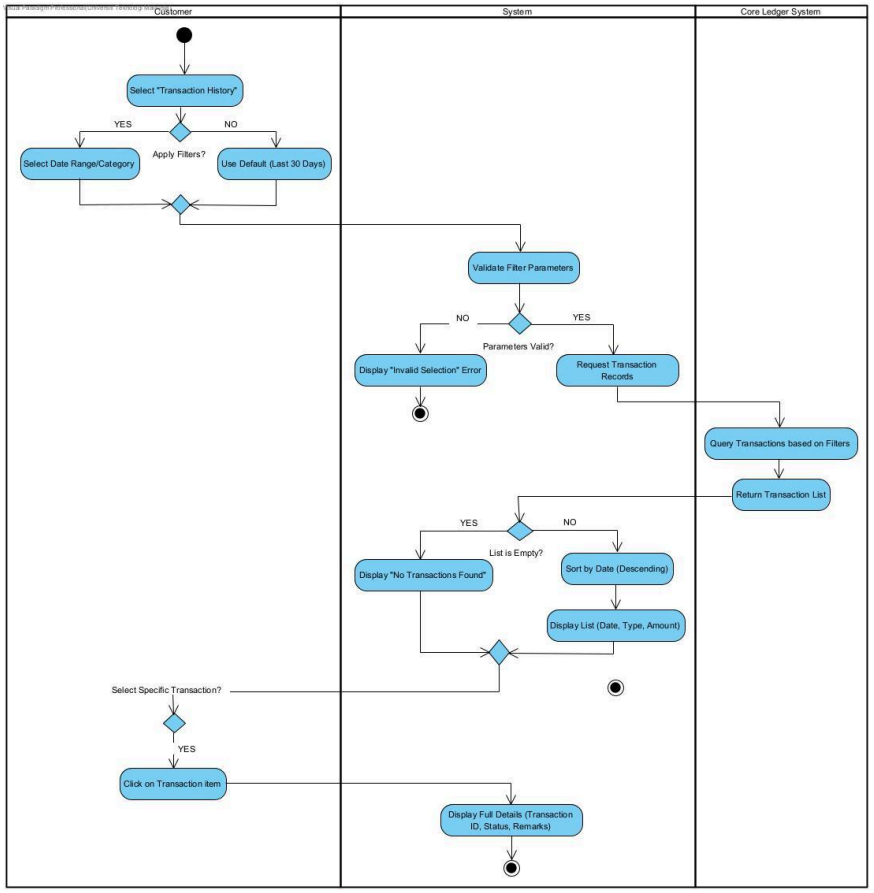


Figure 3.2.1.2: View Transaction History

For UC003, this diagram outlines the process for users to maintain financial transparency. It shows the interaction between the Etiqa+ app and the Core Ledger System. Users can apply custom filters to sort through payouts, reinvestments, and withdrawals. The system validates these parameters and retrieves accurate real-time data, allowing the user to drill down into specific transaction IDs and status remarks for auditing purposes.

3.2.1.3 UC016 Download Policy Certificate

History Log:	1.0 (22/12/2025)	Initial Draft	
	1.1 (01/01/2026)	Updated …..	
Version:	1.0		
Use Case ID:	UC016		
Use Case Name:	Download Policy Certificate		
Created By:	Mohammed Alsakkaf	**Last Updated By:**	Mohammed Alsakkaf
Date Created:	01 January 2026	**Last Revision Date:**	01 January 2026
Actors:	Customer, Policy Backend Services		
Description:	Allows the user to generate, view, and save a digital PDF copy of their active insurance or Takaful certificate		
Pre-conditions:	1. The user is successfully logged into the Etiqa+ app. 2. User has an active policy selected in their policy portfolio		
Normal Flow:	1. The user clicks "Download Certificate" on an Active Policy. 2. System validates the current user session. 3. The system requests the Policy Data and PDF Template from the Backend. 4. Policy Backend Services retrieves Policy Metadata from the database. 5. Policy Backend Services fetches raw certificate data. 6. The system loads the Certificate PDF Template. 7. The system populates the template with user and policy data. 8. The system generates the final PDF document. 9. The system prepares the download headers. 10. User receives the file download prompt		
Alternative Flow:	A1: Session Expired (At Step 2)		

	1. If the session is no longer valid, the system displays "Session Expired" error. 2. Use Case ends and redirects the user to Login (UC001).		
Exception:	E1: Ineligible for Download (At Step 4) 1. If Policy Backend Services determines the policy is not eligible for download (e.g., status is Expired), the system displays error. 2. Use case terminates.		
Post-conditions:	A digital certificate is successfully generated and stored on the user's device.		
Related Requirement:	**ID**	**Requirement**	**Priority**
	FR-UC016-01	The System **SHALL** allow users to generate and download digital policy certificates in PDF format for offline access.	**High**
	FR-UC016-02	The System **SHALL** verify that the user session is active and the policy is eligible for download before initiating the generation process.	**High**

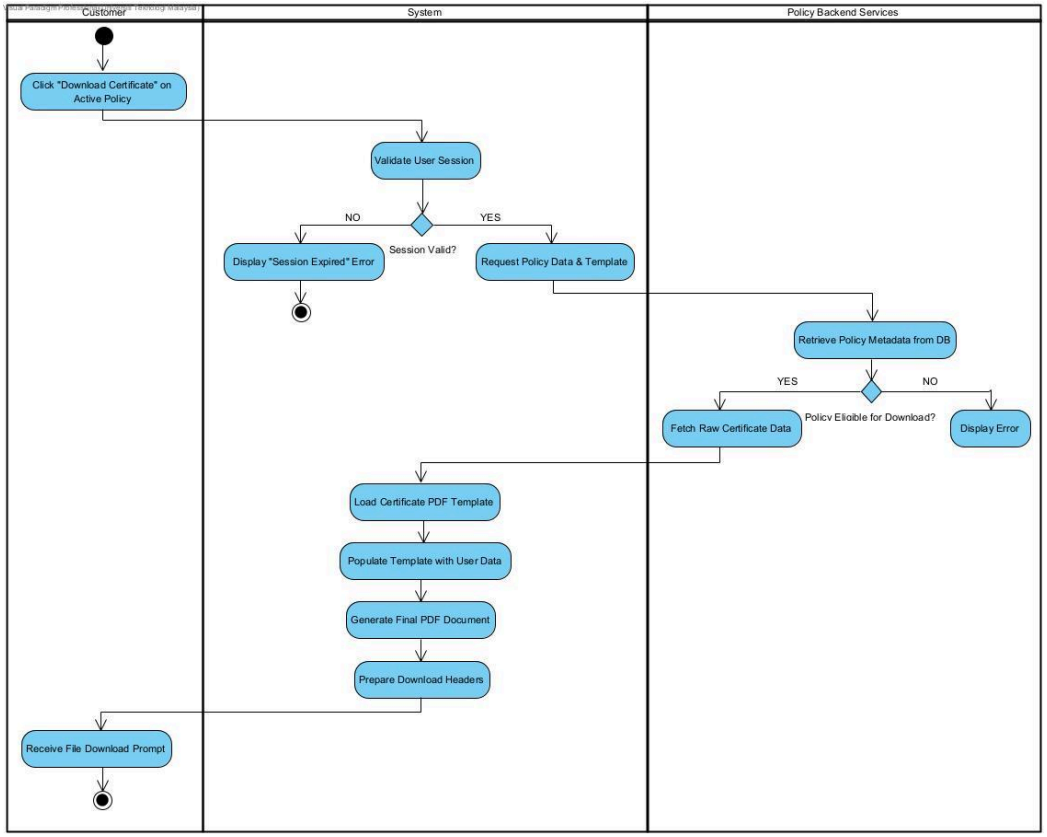


Figure 3.2.1.3: UC016 Download Policy Certificate

For UC016, this diagram details the sub-process for retrieving digital proof of coverage. Once the user selects an active policy, the system coordinates with the Policy Backend Services to fetch the necessary metadata and raw certificate data. The process concludes with the generation of a secure PDF document and a file download prompt, providing the user with instant access to their official documents without needing manual physical copies.

3.2.1.4 MUC01 Account Takeover

History Log:	1.0 (22/12/2025)	Initial Draft	
	1.1 (01/01/2026)	1.0 - Initial Misuse Specification	
Version:	1.0		
Misuse Case ID:	MUC01		
Use Case Name:	Account Takeover		
Created By:	Mohammed Alsakkaf	**Last Updated By:**	Mohammed Alsakkaf
Date Created:	01 January 2026	**Last Revision Date:**	01 January 2026
Actors:	Attacker		
Description:	A scenario where an attacker obtains legitimate credentials to gain unauthorized access to a user's account.		
Threat:	Unauthorized access to sensitive financial data and ability to initiate fraudulent transactions.		
Pre-conditions:	Attacker has obtained valid credentials (via phishing, data leak, etc.).		
Misuse Flow:	1. Attacker navigates to Etiqa+ Login Page 2. Attacker enters stolen Username and Password. 3. System validates credentials. 4. System grants session token. 5. The system loads the Customer Dashboard. 6. Attacker accesses Account Balance and Transaction History.		
Alternative Flow:	A1: Partial Data Access 1. Attacker chooses to only access View Account Balance' (UC002) and skips 'View Transaction History' (UC003). 2. Use case proceeds to unauthorized action steps.		

<table>
<tr><td>Exception:</td><td colspan="3">E1: Password Incorrect

1. If the attacker enters the wrong password, the system displays an error and access is denied.</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">Attacker gains full control of the user's session and private data.</td></tr>
<tr><td>Related User Case(s):</td><td colspan="3">Threatens: UC001 Login.Mitigated By: SC001 Block Attack.</td></tr>
<tr><td rowspan="3">Security Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>SR-MUC01-01</td><td>The System SHALL identify and flag concurrent logins from suspicious or unrecognized IP ranges.</td><td>Critical</td></tr>
<tr><td>SR-MUC01-02</td><td>The System SHALL generate a security log entry for every failed login attempt associated with a potential takeover event.</td><td>High</td></tr>
</table>

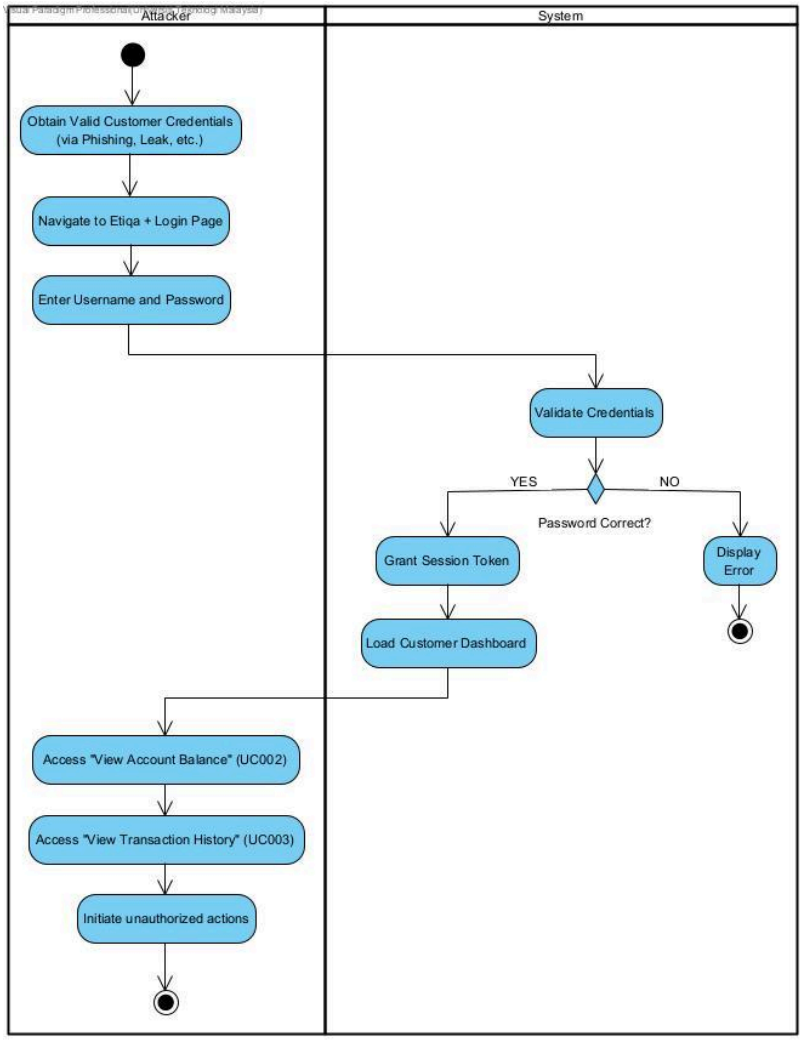


Figure 3.2.1.4: MUC01 Account Takeover

For MUC01, this misuse case diagram visualizes a critical security threat where an external attacker attempts to gain access using stolen credentials obtained via phishing or data leaks. The diagram illustrates how an attacker bypasses basic security if multi-factor authentication is not present, highlighting the system's vulnerability to unauthorized data retrieval and fraudulent actions if the session is granted to a non-legitimate user.

3.2.1.5 SC001 Block Attack (Security Mitigation)

History Log:	1.0 (22/12/2025)	Initial Draft	
	1.1 (01/01/2026)	Updated …..	
Version:	1.0		
Use Case ID:	SC001		
Use Case Name:	Block Attack (Security Mitigation)		
Created By:	Mohammed Alsakkaf	**Last Updated By:**	Mohammed Alsakkaf
Date Created:	01 January 2026	**Last Revision Date:**	01 January 2026
Actors:	Attacker, System, Customer (Victim)		
Description:	A security sub-process that detects unauthorized login attempts from unrecognized devices and blocks access.		
Pre-conditions:	1. User is attempting to log in from a device with a different hardware ID or IP geo-location.		
Normal Flow:	1. System detects "Correct Password" from an unrecognized device/location. 2. System triggers UC012 Security Challenge. 3. System generates One-Time Password (OTP). 4. System sends OTP to the registered mobile number of the Customer (Victim). 5. System displays "Enter OTP" screen. 6. Attacker is prompted for OTP 7. System blocks access when the valid OTP is not provided..		
Alternative Flow:	A1: Device Authorization 1. If valid OTP is entered, system authorizes the device. 2. System proceeds to Step 4 of Login (UC001) Normal Flow.		

<table>
<tr><td>Exception:</td><td colspan="3">E1: Access Denied (At Step 6)

1. If invalid code entered or timeout: System fails authentication.

2. System locks account temporarily.

3. System logs "Security Incident" in audit logs.

4. System displays "Access Denied" to the Attacker.</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">Unauthorized access is blocked, and the incident is recorded for compliance.</td></tr>
<tr><td rowspan="3">Related Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>SR-SC001-01</td><td>The system SHALL block logins from unrecognized devices until a secondary challenge is passed</td><td>Critical</td></tr>
<tr><td>SR-SC001-02</td><td>The System SHALL automatically log a security incident report when a secondary challenge (OTP) fails.</td><td>High</td></tr>
</table>

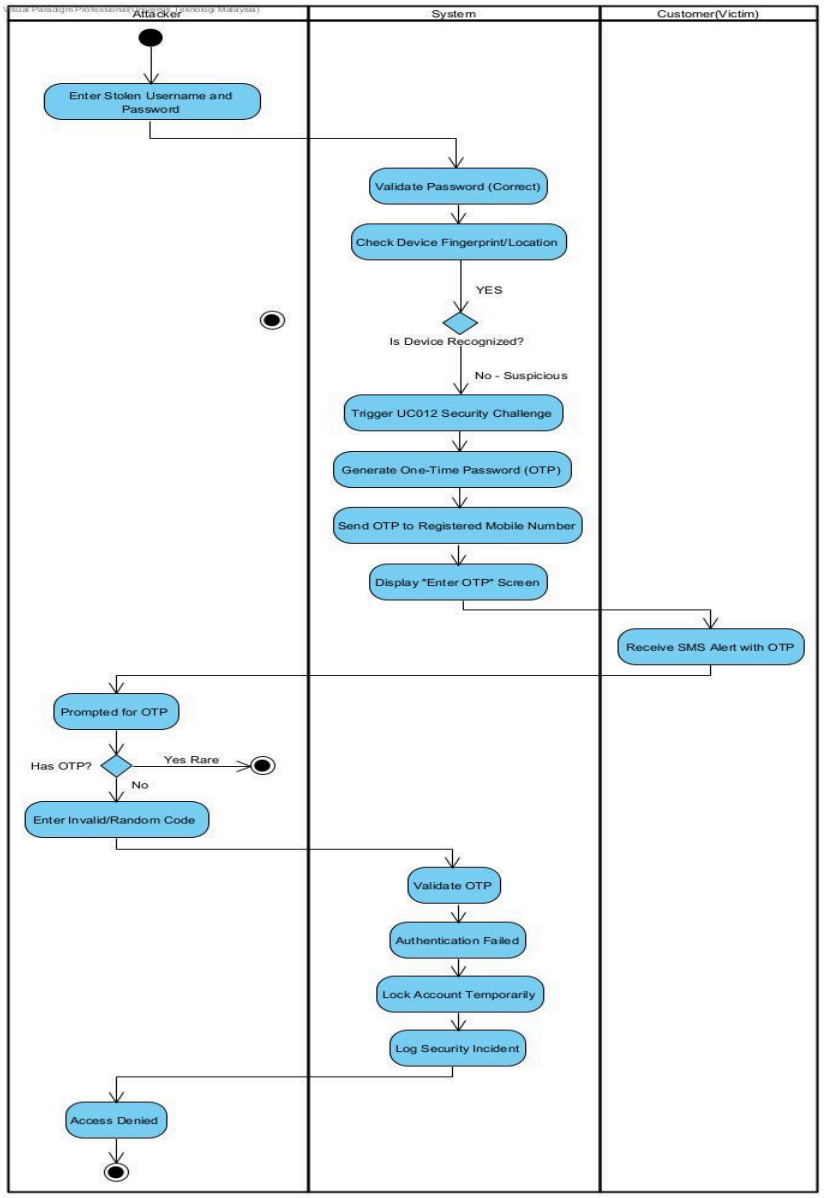


Figure 3.2.1.5: SC001 Block Attack

For SC001, this diagram illustrates the defensive mitigation logic designed to counter account takeover attempts. The system proactively checks the device fingerprint and geo-location during every login. If the device is unrecognized, the system triggers a secondary security challenge (OTP). This ensures that even if an attacker has the correct password, they are blocked from entry, and the legitimate customer is notified of the suspicious activity.

3.2.2 Module 2: Claims Processing

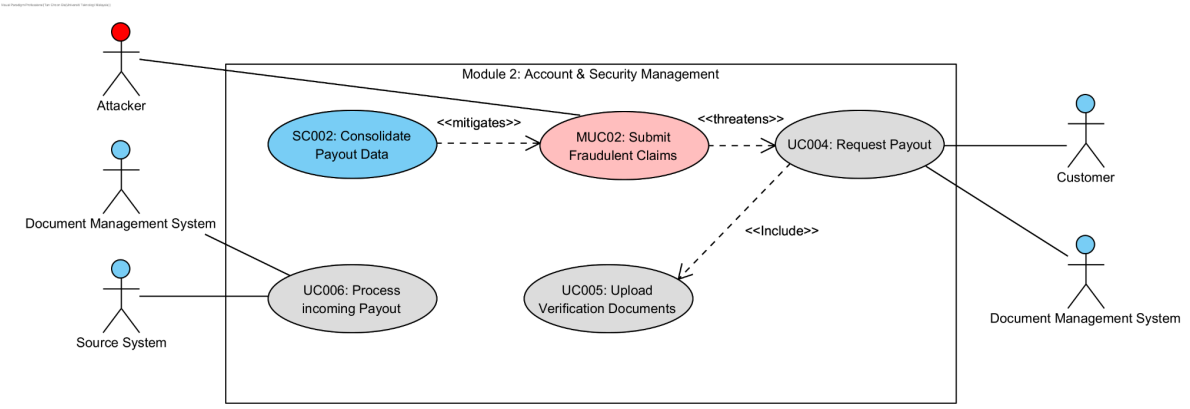


Figure 3.2.2: Use Case Diagram for Module 2

3.2.2.1 UC004 Request Payout

History Log:	1.0 (22/12/2025)	Initial Draft	
	1.1 (29/12/2025)	Updated to include UC005 integration.	
Version:	1.1		
Use Case ID:	UC004		
Use Case Name:	Request Payout		
Created By:	Tan Choon Da	**Last Updated By:**	Tan Choon Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Customer, Etiqa+ System		
Description:	Allows the Customer to initiate a claim payout request for an eligible policy, enter the claim amount, and upload necessary verification documents.		
Pre-conditions:	1. The customer is logged into the Etiqa+ App. 2. Customers have at least one eligible policy with claimable status.		
Normal Flow:	1. Customer navigates to "Claims & Payout" menu. 2. Customer selects an eligible Policy/Claim from the list.		

	3. The system retrieves policy details and displays payout options. 4. Customer enters the Claim Amount. 5. Customer clicks "Confirm Details". 6. The system checks document requirements. 7. System triggers UC005: Upload Verification Documents (Include Relationship). 8. The system receives validation confirmation from the document upload process. 9. The system generates a Request ID. 10. The system sets status to "Pending Processing". 11. The system displays a business success message to the Customer.
Alternative Flow:	A1: Invalid Document (During Step 8) 1. If the document validation (from UC005) returns "False": 2. System displays error "Missing/Invalid Documents". 3. Use Case ends (Customer must retry).
Exception:	E1: System Error 1. If the System fails to generate a Request ID at Step 9: 2. System logs the error. 3. The system displays "Service Unavailable, please try again later."
Post-conditions:	A new Payout Request is created in the database with status "Pending Processing".

Related Requirement:	**ID**	**Requirement**	**Priority**
	FR-UC004-01	The System **SHALL** allow the Customer to select an eligible policy and input a claim amount.	**Basic**
	FR-UC004-02	The System **SHALL** invoke the Document Upload process (UC005) before finalizing the request.	**Basic**

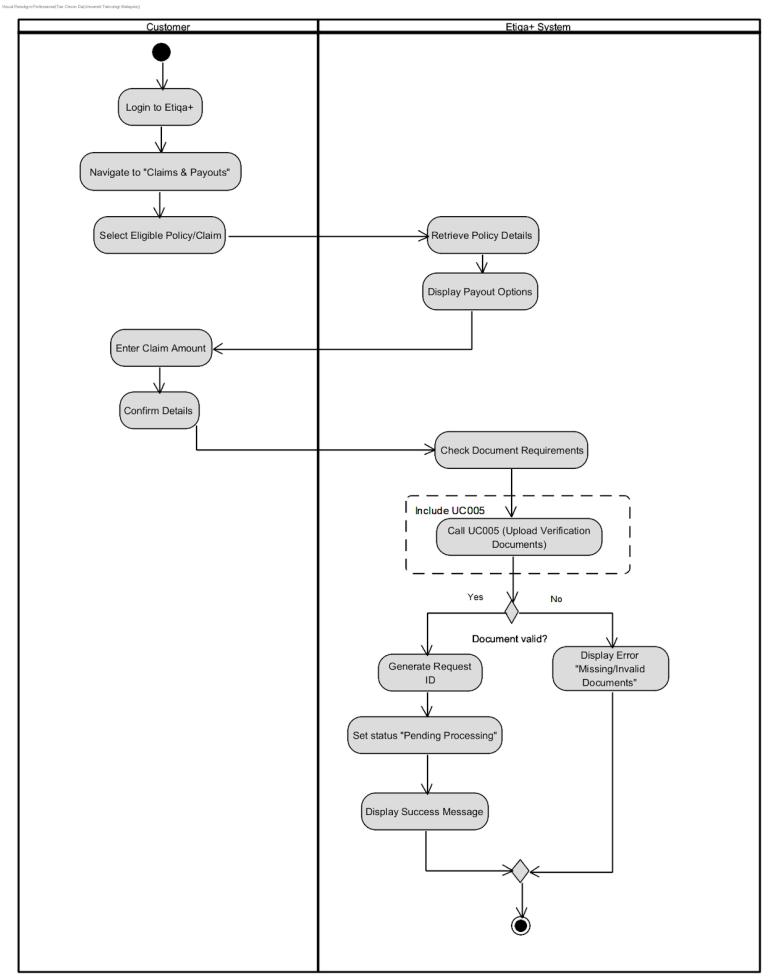


Figure 3.2.2.1: Activity Diagram for UC004 Request Payout

For UC004, this diagram illustrates how the user initiates the claiming payout process. Firstly, the flow begins with the customer logging into the Etiqa+ App, navigating to “Claims and Payout” and selecting an eligible policy. After that, the system will retrieve the policy details and display the payout options. Then, the customer needs to enter the claim amount and confirm the details. The system will check the document requirement by calling the UC005 (Upload Verification Documents). If the document is successfully uploaded and the request format is valid, the system will generate a request ID and set the status to “Pending”, concluding with a success message to the user. If the document is not valid, the system will display an error to the customer and end the request payout process.

3.2.2.2 UC005 Upload Verification Document

History Log:	1.0 (22/12/2025)	Initial Draft	
Version:	1.0		
Use Case ID:	UC005		
Use Case Name:	Upload Verification Document		
Created By:	Tan Choon Da	**Last Updated By:**	Tan Choon Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Customer, Document Management System		
Description:	Handles the selection, validation, malware scanning, and storage of supporting documents for a payout request.		
Pre-conditions:	1. UC004 (Request Payout) has been initiated. 2. The "Upload Document" prompt is active.		
Normal Flow:	1. Customer selects the Document Type. 2. The customer browses and selects a file from their device. 3. Document Management System (DMS) receives the file. 4. DMS validates the file type and size. 5. DMS performs a malware scan. 6. DMS stores the document securely. 7. DMS links the document to the current Payout Request. 8. DMS returns a "Success" signal to the parent process.		
Alternative Flow:	A1: File Validation Failed (At Step 4) 1. If the file type or size is invalid: 2. DMS rejects the file. 3. DMS displays error "Invalid File".		

<table>
<tr><td>History Log:</td><td>1.0 (22/12/2025)</td><td colspan="2">Initial Draft</td></tr>
<tr><td></td><td colspan="3">4. Return to Step 1.</td></tr>
<tr><td>Exception:</td><td colspan="3">E1: Malware Detected (At Step 5)

1. If the scan detects malware:

2. DMS immediately quarantines/deletes the file.

3. DMS logs a security alert.

4. DMS displays "Security Risk Detected" error.

5. Use Case ends.</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">The document is successfully stored and linked to the Payout Request ID.</td></tr>
<tr><td rowspan="3">Related Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>SR-UC005-01</td><td>The System SHALL scan all uploaded files for malware before storage.</td><td>Basic</td></tr>
<tr><td>FR-UC005-01</td><td>The System SHALL validate that files meet the required format (PDF/JPG) and size limits.</td><td>Excitement</td></tr>
</table>

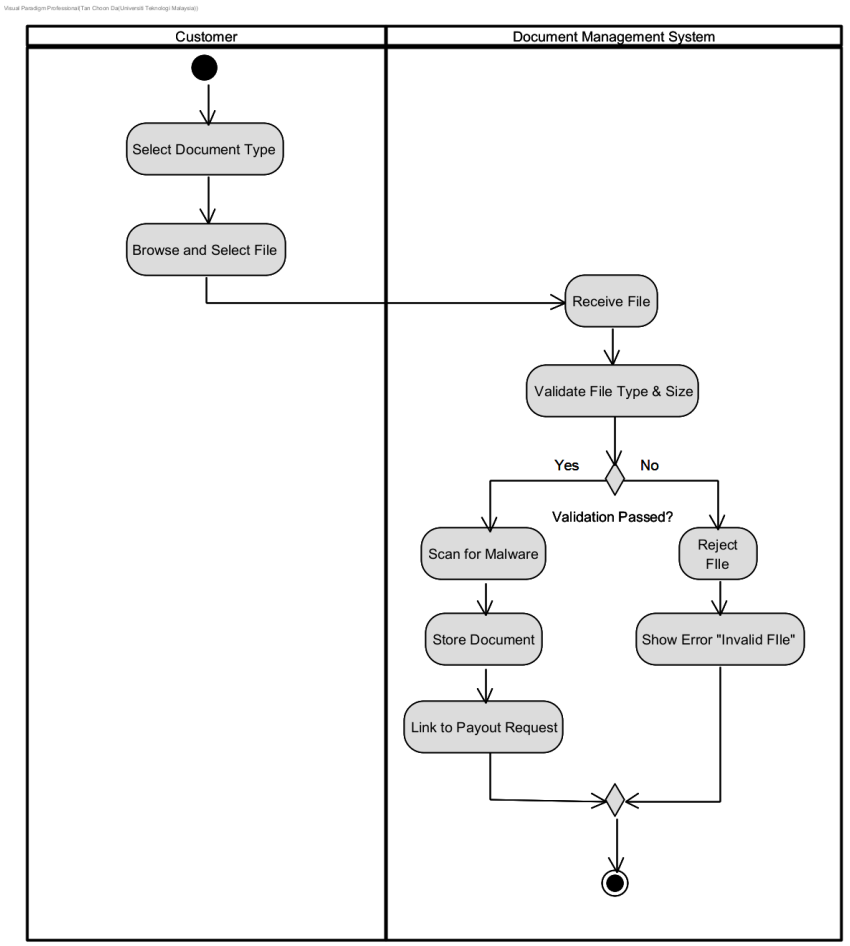


Figure 3.2.2.2: Activity Diagram for UC005 Upload Verification Document

For UC005, this diagram details the sub-process for handling file upload, which is invoked by the parent use case (UC004). The customer needs to select document type, browse and select the related file. After that, the system will receive the file and validate the file type and size. If any validation fails, the file is rejected with an error message. If all validation passes, the system will perform a malware scanning, document is securely stored and linked to the specific payout request ID , ensuring the claim has the necessary evidence for processing.

3.2.2.3 UC006 Process Incoming Payout

History Log:	1.0 (22/12/2025)	Initial Draft	
Version:	1.0		
Use Case ID:	UC006		
Use Case Name:	Process Incoming Payout		
Created By:	Tan Choon Da	**Last Updated By:**	Tan Choon Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Source System, Etiqa+ System		
Description:	Orchestrates the backend approval of a payout by validating the policy status and funds before crediting the user's wallet.		
Pre-conditions:	1. A Payout Request has been triggered by the Source System. 2. System connectivity to Core Ledger is active.		
Normal Flow:	1. Source System triggers payout data. 2. Etiqa+ System receives the payout instruction. 3. System validates the Policy Status. 4. System confirms Policy is Active AND Funds are Available. 5. The system approves the Payout Transaction. 6. System logs transaction details. 7. System triggers UC014: Update User Balance. 8. The system sends a notification to the Customer.		
Alternative Flow:	A1: Validation Failed (At Step 4) 1. If Policy is Inactive OR Funds are Unavailable: 2. System rejects the Payout. 3. System logs the Error Reason.		

<table>
<tr><td></td><td colspan="3">4. Use Case ends.</td></tr>
<tr><td>Exception:</td><td colspan="3">E1: Update Balance Failure (At Step 7)

1. If UC014 fails (e.g., database lock):

2. The system rolls back the transaction.

3. System logs "Transaction Failed" status.</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">The Payout Transaction is recorded as "Approved" (or "Rejected"), and the user is notified.</td></tr>
<tr><td rowspan="3">Related Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>FR-UC006-01</td><td>The System SHALL validate policy status and fund availability before approving any payout.</td><td>Basic</td></tr>
<tr><td>FR-UC006-02</td><td>The System SHALL automatically trigger a balance update (UC014) upon approval.</td><td>Basic</td></tr>
</table>

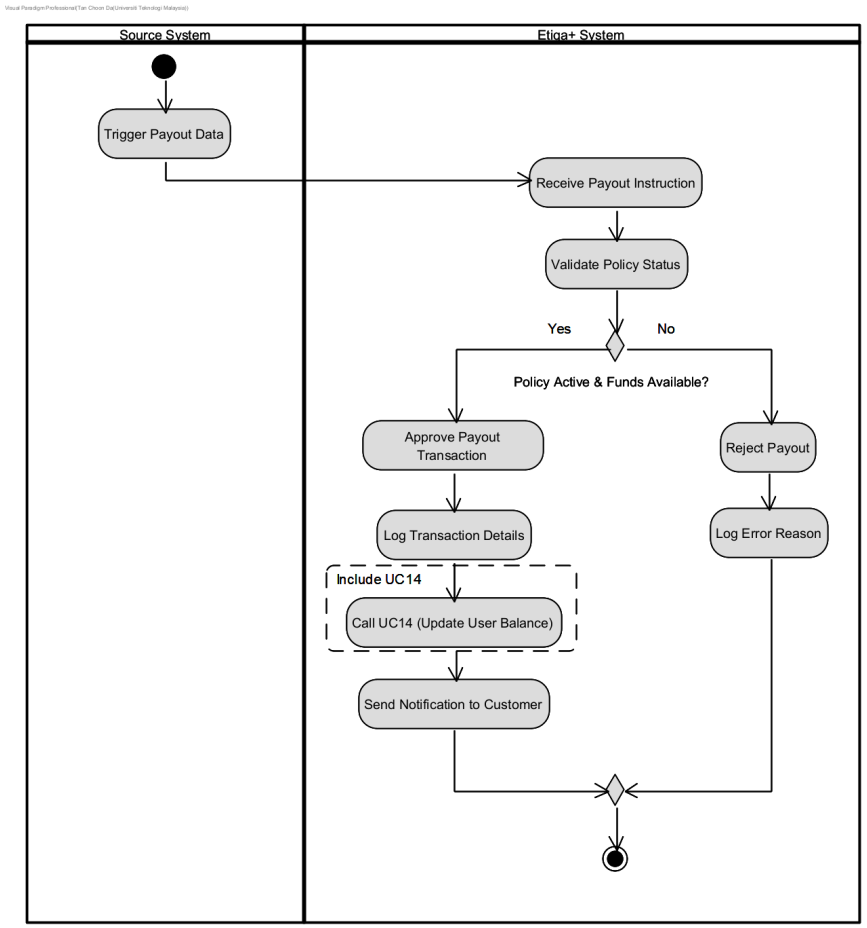


Figure 3.2.2.3: Activity Diagram for UC006 Process Incoming Payout

For UC006, this diagram represents the backend orchestration triggered by the Source System. Once a payout instruction is received, the Etiqa+ System validates the policy status and fund availability. If the policy is active and funds are available, the system will approve the transaction and explicitly calls UC14 (Update User Balance) to credit the customer's eWallet. Finally, a notification will be sent to the customer. If the policy is inactive or the funds are unavailable, the system will reject the payout and log the error reason for audit purposes.

3.2.2.4 MUC02 Submit Fraudulent Claim

History Log:	1.0 (22/12/2025)	Initial Draft	
Version:	1.0		
Use Case ID:	MUC02		
Use Case Name:	Submit Fraudulent Claim		
Created By:	Tan Choon Da	**Last Updated By:**	Tan Choon Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Misuser(s)/Threat Agents(s):	Attacker, Payout Service System		
Description:	A scenario where an attacker attempts to submit a claim with manipulated amounts or forged documents, and how the system mitigates it.		
Threat:	The system is vulnerable to financial loss if it blindly accepts user inputs without validating them against the "Source of Truth" (Legacy Database).		
Triggers	The Attacker clicks the "Submit Payout Request" button after entering manipulated data.		
Pre-conditions:	1. Attacker has valid login credentials. 2. Attackers have access to the "Request Payout" feature.		
Misuse Flow (Attack Steps):	1. Attacker logs into Etiqa+. 2. Attacker navigates to "Request Payout". 3. Attacker inputs a Manipulated Claim Amount (e.g., higher than the allowable policy limit). 4. Attacker uploads Forged/Altered Documents to support the fake claim. 5. Attacker submits the Payout Request.		

<table>
<tr><td>Mitigation Flow:</td><td colspan="3">1. Attacker logs into Etiqa+.

2. Attacker navigates to "Request Payout".

3. Attacker inputs a Manipulated Claim Amount (e.g., higher than the allowable policy limit).

4. Attacker uploads Forged/Altered Documents to support the fake claim.

5. Attacker submits the Payout Request.</td></tr>
<tr><td>Exception Flows:</td><td colspan="3">E1: Attack Successful (False Negative)

1. The system marks verification as PASSED.

2. The system approves the transaction.

3. The system transfers funds to the Attacker.

4. Attacker receives funds (System fails to protect asset).</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">If Mitigated: Transaction is Rejected; User Account is Locked; Incident is Logged

If Successful: Funds are deducted from the ledger erroneously.</td></tr>
<tr><td>Related User Case(s):</td><td colspan="3">Threatens: UC004 Request Payout.

Mitigated By: SC002 Consolidate Payout Data.</td></tr>
<tr><td rowspan="3">Security Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>SR-MUC02-01</td><td>The System SHALL validate every payout request against the Legacy Database (Source System) to ensure the claim amount does not exceed the policy limit.</td><td>Basic</td></tr>
<tr><td>SR-MUC02-02</td><td>The System SHALL automatically lock the user account if a data mismatch indicative of fraud is detected during reconciliation.</td><td>Basic</td></tr>
</table>

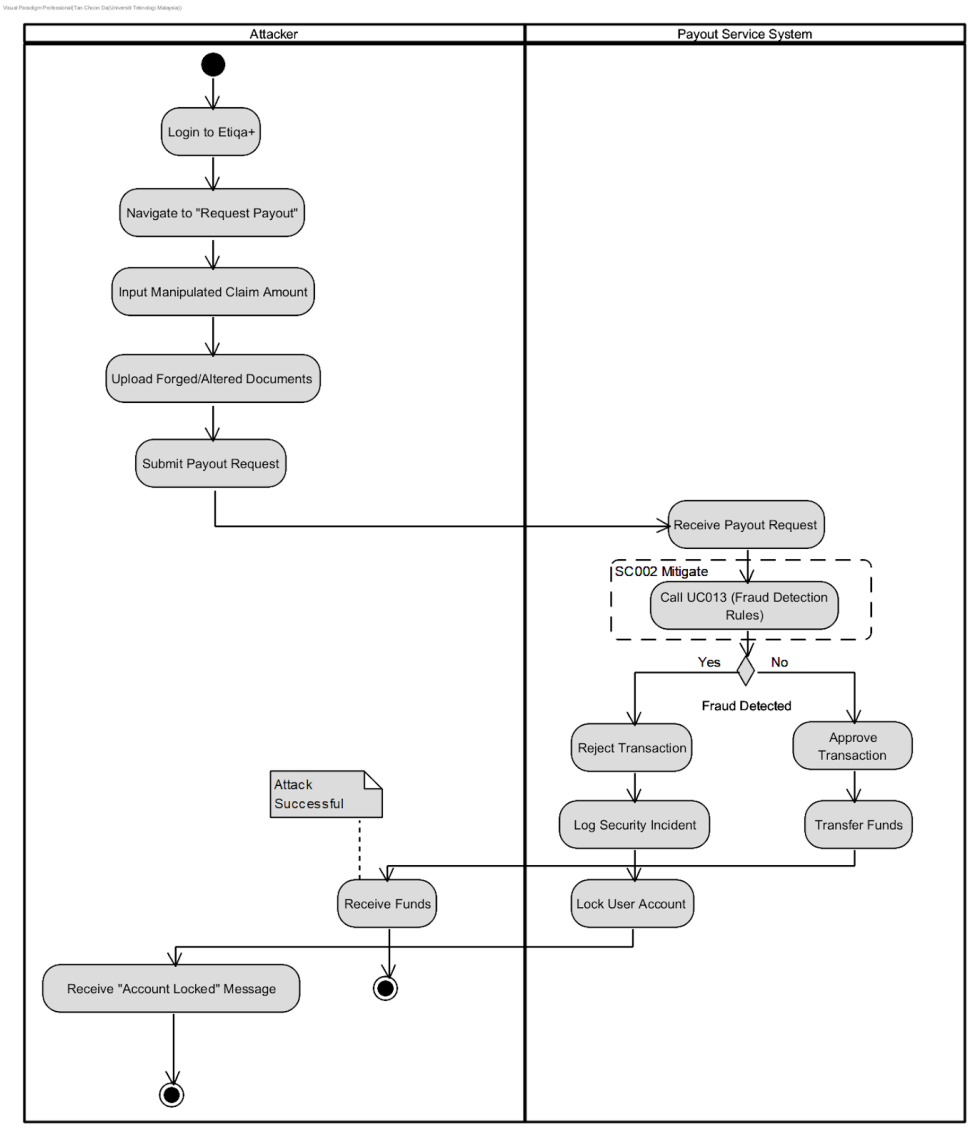


Figure 3.2.2.4: Activity Diagram for MUC02 Submit Fraudulent Claim

For MUC02, this diagram visualizes the "threat" scenario from the perspective of an attacker. The attacker attempts to exploit the system by inputting manipulated claim amounts and uploading forged documents via the "Request Payout" interface. The system counters this by triggering the fraud detection rules by utilizing UC013. If the fraud is detected, the transaction is rejected, the security incident is logged and the user account will be locked. If the system fails to detect the discrepancy, then it will approve the transaction for the attacker and the attacker will receive the transaction.

3.2.2.5 SC002 Consolidate Payout Data

History Log:	1.0	Initial Draft	
Version:	1.0		
Use Case ID:	SC002		
Use Case Name:	Consolidate Payout Data (Security Case)		
Created By:	Tan Choon Da	**Last Updated By:**	Tan Choon Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Payout Service System, Data Reconciliation Service		
Description:	A security process that verifies incoming payout requests by reconciling them against the Legacy Database (Source of Truth) to prevent fraud.		
Pre-conditions:	1. A Payout Request has been initiated (via MUC02 or UC004). 2. Connection to the Legacy Database is active.		
Normal Flow:	1. Payout Service System requests data verification, providing the "Incoming Request Object". 2. Data Reconciliation Service parses the incoming request data. 3. Service queries the Source System (Legacy Database). 4. Service retrieves the original Policy Limit and Status. 5. Service maps legacy data to the "To-Be" format. 6. Service compares [Incoming Amount] vs [Policy Limit]. 7. Service confirms NO discrepancy found. 8. Service marks verification as PASSED. 9. Service returns the result to Payout Service System.		
Alternative Flow:	A1: Discrepancy Found (Fraud Detection) (At Step 6)		

	1. If [Incoming Amount] > [Policy Limit] or data mismatch: 2. Service generates a "Data Mismatch Flag". 3. Service marks verification as FAILED. 4. Service returns "Fail" result (Triggers Rejection in Parent Flow).		
Exception:	E1: Database Timeout 1. If Source System is unreachable at Step 3: 2. Service logs "Reconciliation Timeout". 3. Service returns "Verification Error" (System defaults to Block Transaction for security).		
Post-conditions:	A Pass/Fail verification result is returned to the Payout Service System.		
Related Requirement:	**ID**	**Requirement**	**Priority**
	SR-SC002-01	The System **SHALL** compare every payout request against the Legacy Database to ensure the claim amount does not exceed the policy limit.	**Basic**

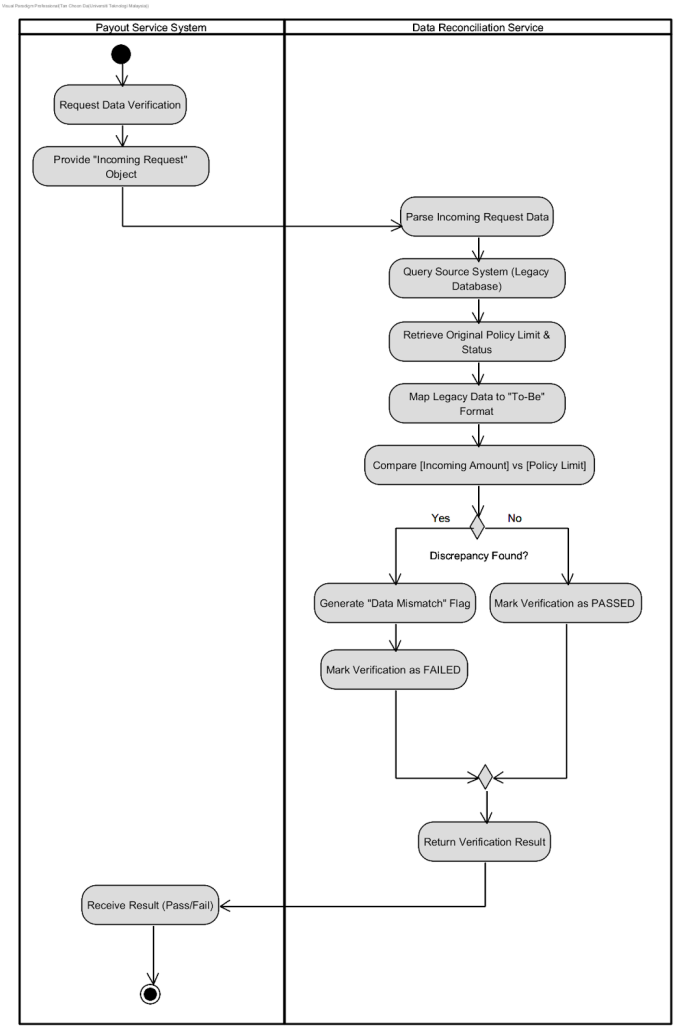


Figure 3.2.2.5: Activity Diagram for SC002 Consolidate Payout Data

For UC013, this diagram explains the way used to mitigate fraud, it shows how the system decides if a claim is fraudulent. The Data Reconciliation Service fetches the "Sources of Truth" from the legacy database and compares it against the "Incoming Request" data. It explicitly checks for discrepancies such as claim amount exceeding the policy limit or mismatched user ID. If the discrepancies are found, the data reconciliation service will generate a "Data Mismatch" flag and mark the verification as "Failed". If there are no discrepancies found, the system will mark the verification as "Passed". After the decision point, the data reconciliation system will return the verification result to the payout service system.

3.2.3 Module 3: Withdrawals & Transfers

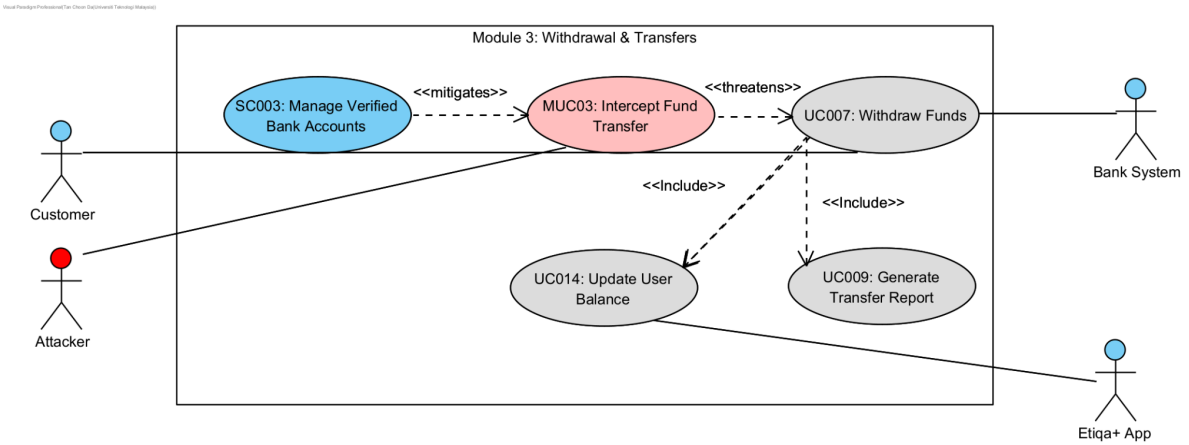


Figure 3.2.3: Use Case Diagram for Module 3

3.2.3.1 UC007 Withdraw Funds

History Log:	1.0	Initial Draft based on Activity Diagrams	
Version:	1.0		
Use Case ID:	UC-007		
Use Case Name:	Withdraw Funds		
Created By:	Gan Rui En	**Last Updated By:**	Gan Rui En
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Customer, Etiqa+ App, Bank System		
Description:	This use case describes the process where a Customer withdraws funds from their Etiqa+ account to a verified external bank account. The system validates the balance, authenticates the request, processes the transaction with the Bank System, updates the wallet balance, and generates a transaction report.		
Pre-conditions:	1. The customer is logged into Etiqa+ App. 2. Customers have at least one verified bank account linked.		
Normal Flow:	**1.** Customer selects "Withdraw Funds" from the interface. **2.** Etiqa System validates the input format. **3.** Etiqa System checks the account balance (Ref: UC002).		

	4. Etiqa System confirms sufficient balance. **5.** Etiqa System retrieves verified bank accounts (Ref: UC006). **6.** Customer selects the destination bank account. **7.** Customer confirms withdrawal details. **8.** Etiqa System authenticates the request via MFA/PIN. **9.** Etiqa System sends a transfer request to the Bank System. **10.** Bank System processes the transaction and returns a Success Code. **11.** Etiqa System triggers "Update Balance" process (Ref: UC014). **12.** Etiqa System triggers "Generate Report" process (Ref: UC009). **13.** Etiqa System displays a success message to the Customer.
Alternative Flow:	**A1. Insufficient Funds** At step 4, if the check for sufficient balance returns **False**: 1. Etiqa System displays "Insufficient Funds" error message. 2. The use case terminates.
Exception:	**E1. Bank Transaction Failed** At step 10, if the Bank System fails to process the transaction (returns **False**): 1. Bank System returns an Error Code. 2. Etiqa System displays "Bank Transfer Failed" message. 3. The use case terminates.
Post-conditions:	1. Funds are transferred to the Customer's bank account. 2. User wallet balance is updated (decreased). 3. A transaction report/receipt is generated.

Related Requirement:	**ID**	**Requirement**	**Priority**

	FR UC-007-01	The system shall be able to validate the input format for the withdrawal amount and bank details	**Basic**
	FR UC-007-02	The system shall be able to check the user's current account balance against the withdrawal amount	**Basic**
	FR UC-007-03	The system shall be able to retrieve and display a list of verified bank accounts for the user to select	**Basic**
	FR UC-007-04	The system shall require the user to authenticate the request via MFA or PIN before processing	**Basic**
	FR UC-007-05	The system shall allow the user to confirm the withdrawal details before sending the request to the bank	**Basic**
	QR UC-007-01	The system shall display an "Insufficient Funds" error message when the wallet balance is lower than the requested amount	**Basic**
	QR UC-007-02	The system shall display a "Bank Transfer Failed" message if the external bank system returns an error code	**Performance**
	CR UC-007-01	The system shall only trigger the balance update and report generation after receiving a "Success" signal from the Bank System	**Performance**

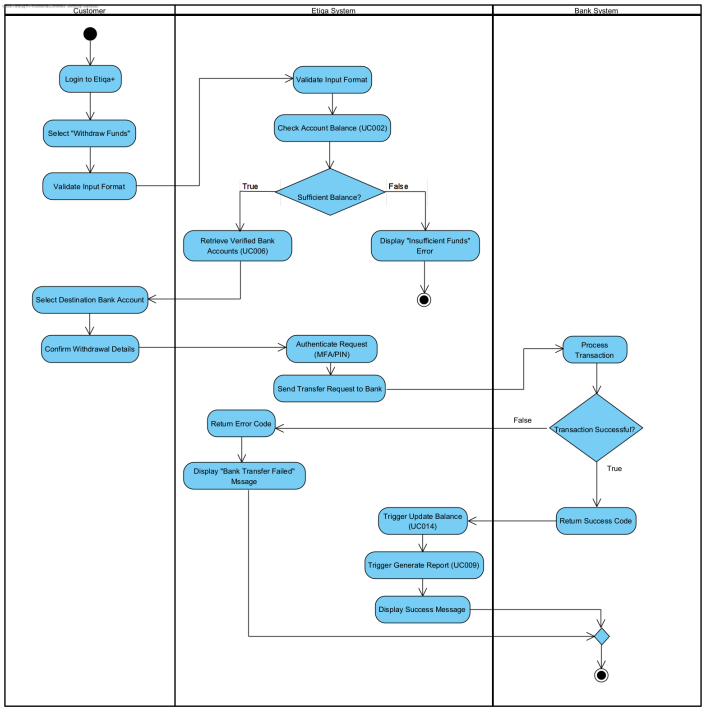


Figure 3.2.3.1: Activity Diagram for UC007 Withdraw Funds

The withdrawal process led to the secure transfer of funds. It begins with the customer requesting an amount, followed by system validation of the account balance. If funds are sufficient, the user selects a pre-verified bank account. The system authenticates the user via MFA before transmitting the request to the external Bank System. Upon a successful bank response, the system triggers background processes to update the user's balance and generate a transaction report, finally displaying a success message to the customer.

3.2.3.2 UC009 Generate Transfer Report

History Log:	1.0	Initial Draft based on Activity Diagrams	
Version:	1.0		
Use Case ID:	UC-009		
Use Case Name:	Generate Transfer Report		
Created By:	Gan Rui En	**Last Updated By:**	Gan Rui En
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Customer, Etiqa+ App		
Description:	This use case describes how the system collates transaction data to generate a PDF or digital receipt and stores it in the history. It allows the customer to view and optionally download the receipt.		
Pre-conditions:	1. A financial transaction has successfully completed and triggered this process.		
Normal Flow:	**1.** System receives a trigger from UC-007. **2.** System collates the relevant transaction details. **3.** System formats the data into a PDF or Digital Receipt. **4.** System stores the record in the Transaction History. **5.** Customer views the Transfer Receipt on their interface. **6.** Customer decides whether to download the receipt. **7.** If **"Yes"**: Customer initiates download, and System provides the PDF. **8.** Use case ends.		
Alternative Flow:	**A1. No Download** At step 6, if the Customer decides not to download (False): 1. The flow terminates without a file download.		

Exception:	-		
Post-conditions:	1. Transaction details are permanently stored in history. 2. A digital receipt is presented to the user.		
Related Requirement:	**ID**	**Requirement**	**Priority**
	FR UC-009-01	The system shall be able to collate transaction details from the completed withdrawal process	**Basic**
	FR UC-009-02	The system shall be able to format the transaction data into a digital receipt or PDF	**Basic**
	FR UC-009-03	The system shall be able to store the transaction record in the Transaction History database	**Basic**
	FR UC-009-04	The system shall allow the Customer to download the generated receipt as a PDF file	**Excitement**
	QR UC-009-01	The system shall automatically display the transfer receipt to the user immediately upon transaction success	**Performance**
	CR UC-009-01	The system shall only generate a receipt if the transaction has been successfully committed to the ledger	**Basic**

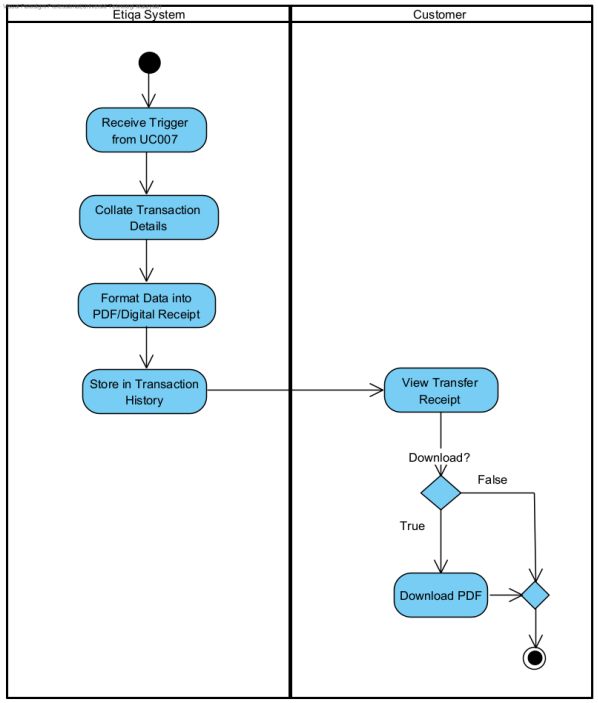


Figure 3.2.3.2: Activity Diagram for UC009 Generate Transfer Report

It serves as a digital receipt mechanism, this process is triggered post-transaction. The system aggregates all critical transaction metadata, including the timestamp, amount, and destination details. This data is formatted into a downloadable PDF or digital view. The generated report is then stored in the customer's transaction history. This allows the user to access, view, and download proof of the transfer at any time, fulfilling the requirement for transparency.

3.2.3.3 UC014 Update User Balance

History Log:	1.0	Initial Draft based on Activity Diagrams	
Version:	1.0		
Use Case ID:	UC-014		
Use Case Name:	Update User Balance		
Created By:	Gan Rui En	**Last Updated By:**	Gan Rui En
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Payout Service System, Database, Etiqa+ App		
Description:	This use case handles the backend logic for updating a user's wallet. It ensures data integrity via concurrency control (locking), updates the ledger, and pushes real-time notifications to the app.		
Pre-conditions:	1. A "Transaction Success" signal is received from the banking/payment gateway.		
Normal Flow:	1. Payout Service System receives "Transaction Success" signal. 2. System locks the user wallet to prevent concurrent modifications. 3. System retrieves the current balance from the Database. 4. System calculates the new balance. 5. System updates the wallet record in the Database. 6. System logs the transaction in the Ledger (Database). 7. System unlocks the user wallet. 8. System sends a Push Notification to the Etiqa+ App. 9. Etiqa+ App refreshes the balance display. 10. Use case ends.		
Alternative Flow:	-		
Exception:	**E1. Database Lock Failure** (Implicit)		

	If Step 2 fails (cannot lock wallet): System retries or logs a concurrency error.		
Post-conditions:	1. User wallet balance is updated in the database. 2. Transaction is logged in the ledger. 3. User is notified via the app.		
Related Requirement:	**ID**	**Requirement**	**Priority**
	FR UC-014-01	The system shall be able to calculate the new balance by deducting the withdrawal amount from the current balance	**Basic**
	FR UC-014-02	The system shall be able to update the user's wallet record in the database with the new balance	**Basic**
	FR UC-014-03	The system shall be able to log the transaction details into the Transaction Ledger	**Basic**
	FR UC-014-04	The system shall be able to push a notification to the Etiqa+ App upon successful update	**Performance**
	CR UC-014-01	The system shall lock the user wallet (Concurrency Control) before retrieving the balance to prevent simultaneous modifications	**Performance**
	QR UC-014-01	The Etiqa+ App shall refresh the balance display automatically without requiring a page reload by the user	**Excitement**

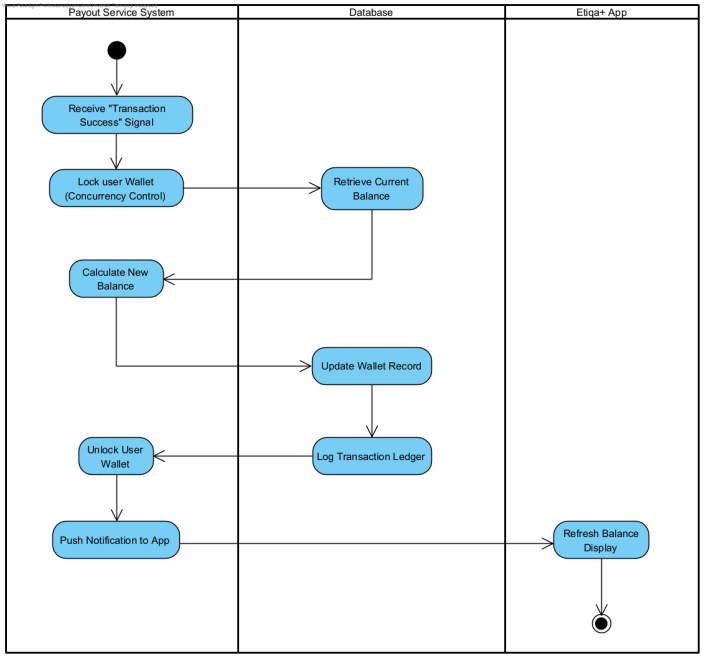


Figure 3.2.3.3: Activity Diagram for UC014 Update User Balance

This automated backend process ensures financial consistency following a transaction. Triggered immediately by a successful withdrawal, the system locks the user's wallet to prevent concurrent modifications. It calculates the new balance by subtracting the withdrawal amount from the current total and updates the database record. Once the ledger is updated, the wallet is unlocked, and a push notification is sent to the customer's application to refresh the displayed balance, ensuring the user sees accurate data in real-time.

3.2.3.4 MUC03 Intercept Fund Transfer

History Log:	1.0	Initial Draft based on Misuse Case Diagram	
Version:	1.0		
Use Case ID:	MUC-003		
Use Case Name:	Intercept Fund Transfer		
Created By:	Gan Rui En	**Last Updated By:**	Gan Rui En
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Misuser(s)/Threat Agents(s):	Attacker		
Description:	This misuse case describes a scenario where an attacker intercepts the network traffic during a legitimate "Withdraw Funds" transaction and attempts to modify the destination account ID to divert funds to an unauthorized account.		
Threat:	Man-in-the-Middle (MitM) / Data Integrity Violation: The attacker compromises the integrity of the transaction data in transit between the client and the server.		
Triggers	A legitimate user initiates a fund transfer/withdrawal request during UC-007.		
Pre-conditions:	1. The user is authenticated and performs a transaction. 2. The attacker has successfully positioned themselves to intercept network traffic..		
Misuse Flow (Attack Steps):	**1.** User initiates the transfer. **2.** Attacker intercepts the traffic generated during UC-007. **3.** Attacker attempts to modify the "Destination Account ID" field in the request payload. **4.** Attacker forwards the modified request to the Etiqa System.		
Mitigation Flow:	1. Etiqa System receives the transfer request. 2. System extracts the Destination Account ID. 3. Mitigation Logic (UC-008): System checks the extracted ID against the User's Verified List. 4. If Account is NOT in Verified List (False):		

<table>
<tr><td></td><td colspan="3">a. System flags the transaction as "Suspicious".

b. System blocks the transaction immediately.

c. System freezes the User Account.

d. System alerts the Compliance Team.

5. Misuse case fails; system is protected.</td></tr>
<tr><td>Exception Flows:</td><td colspan="3">E1. False Negative (Mitigation Failure):

If the attacker manages to modify the ID to another account that is coincidentally on the verified list or bypasses the check, the process moves to "Proceed with Bank Transfer".</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">If Attack Blocked: Transaction is rejected, User account is frozen, and Compliance team is notified.

If Attack Successful: Funds are transferred to the attacker's account.</td></tr>
<tr><td>Related User Case(s):</td><td colspan="3">UC-007: Withdraw Funds (Target)

UC-008: Mitigation Logic (Defense)</td></tr>
<tr><td rowspan="4">Security Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>SR MUC-003-01</td><td>Whitelist Validation: The system must validate the "Destination Account ID" in every transfer request against the user's pre-approved "Verified Beneficiary List" (UC-008) before authorizing the transaction.</td><td>Basic</td></tr>
<tr><td>SR MUC-003-02</td><td>Transaction Blocking: The system must automatically block and reject any transaction where the destination account ID does not match an entry in the verified list.</td><td>Basic</td></tr>
<tr><td>SR MUC-003-03</td><td>Automated Account Freeze: Upon detecting a mismatch (indicating potential tampering), the system must immediately freeze the user's account to prevent further unauthorized attempts.</td><td>Basic</td></tr>
</table>

	SR MUC-003-04	Compliance Alerting: The system must generate a real-time "Suspicious Activity" alert and forward the incident details to the Compliance Team for manual investigation.	**Basic**
	SR MUC-003-05	Integrity Check: The system should ensure the data payload (specifically Account ID and Amount) has not been altered in transit, potentially using cryptographic signatures (Implicit in "Intercept Traffic" threat).	**Basic**

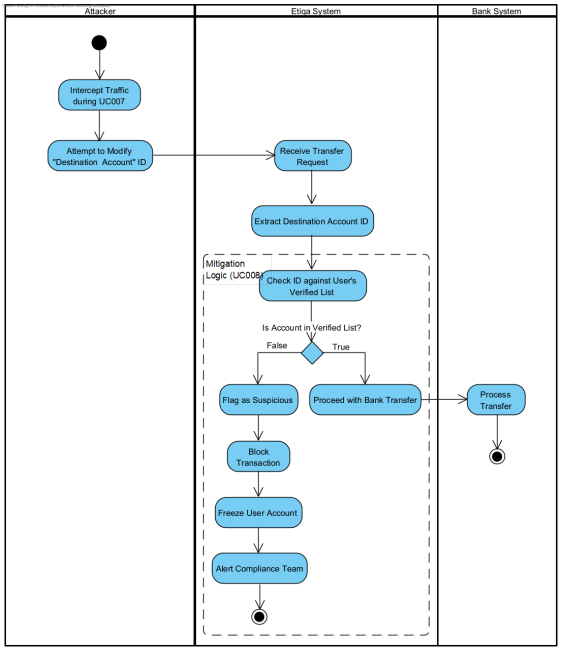


Figure 3.2.3.4: Activity Diagram for MUC03 Intercept Fund Transfer

This diagram illustrates a security threat where an attacker attempts to intercept a withdrawal request and alter the destination account ID. The mitigation logic embedded in the system acts as a firewall. Before processing any transfer, the system cross-references the requested destination ID against the user's pre-approved "Verified List." If the account is not found in this safe list, the transaction is immediately blocked, the user's account is frozen, and a security alert is logged in order to effectively neutralize the theft attempt.

3.2.3.5 SC003 Manage Verified Bank Accounts

History Log:	1.0	Initial Draft based on Activity Diagrams	
Version:	1.0		
Use Case ID:	SC-003		
Use Case Name:	Manage Verified Bank Accounts		
Created By:	Gan Rui En	**Last Updated By:**	Gan Rui En
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Customer, Etiqa System, Bank System		
Description:	This use case describes how a Customer manages their bank accounts within the Etiqa System. It covers adding a new account which requires KYC and external bank verification and removing an existing account.		
Pre-conditions:	1. The Customer has logged in and accessed the "Payment Methods" section.		
Normal Flow:	**1.** Customer selects "Add New Bank Account". **2.** Customer enters Bank Name and Account Number. **3.** Customer uploads or provides Verification Proof. **4.** Etiqa System validates the bank format. **5.** Etiqa System performs Identity Check (KYC Check). **6.** Etiqa System requests account existence verification from Bank System. **7.** Bank System verifies account existence. **8.** Verification is **Successful**. **9.** Etiqa System adds account to Verified List. **10.** Etiqa System notifies the user of success. **11.** Use case ends.		
Alternative Flow:	A1. Remove Account 1. At the start, Customer selects "Select Existing Account" instead of adding new.		

	2. Customer clicks "Remove". 3. Customer confirms the action. 4. Etiqa System removes the record from the database. 5. Etiqa System notifies the user. 6. Use case ends.
Exception:	**E1. Verification Failed** At step 8 of Normal Flow, if verification is **False**: **1.** Etiqa System rejects the addition. **2.** Etiqa System logs the failed verification. **3.** Etiqa System notifies the user of the failure. **4.** Use case terminates.
Post-conditions:	1. A new verified bank account is added to the user's list (if successful). 2. An existing bank account is removed from the user's list (if removed).

Related Requirement:	**ID**	**Requirement**	**Priority**
	FR SC-03-01	**Input Capture:** The system shall provide an interface for the user to enter Bank Name, Account Number, and upload verification proofs (images/documents).	**Basic**
	FR SC-03-02	**Format Validation:** The system shall validate the format of the entered bank account number (e.g., length, numeric only) before processing the request to minimize API errors.	**Basic**
	FR SC-03-03	**Identity Verification (KYC):** The system must perform a Know Your Customer (KYC) check to ensure the name on the bank account matches the registered Etiqa+ user's legal name.	**Basic**
	FR SC-03-05	**Security Logging:** The system shall automatically log all failed verification attempts into the system database for security auditing and fraud detection.	**Basic**

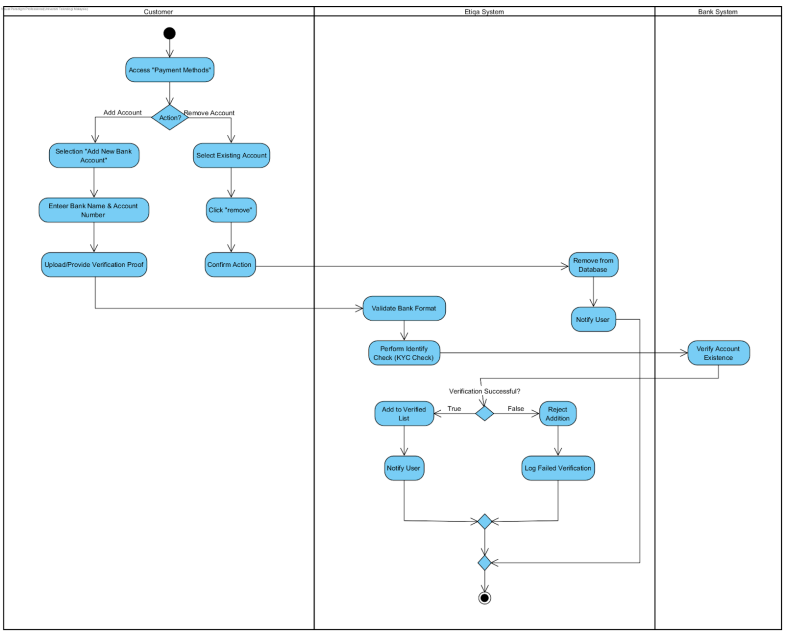


Figure 3.2.3.5: Activity Diagram for SC003 Manage Verified Bank Accounts

This mitigation use case manages the security of external destinations. When a customer attempts to add a new bank account, the system performs a rigid validation process, including format checks and identity verification (KYC) with the external bank. Only if these checks pass then the account transitions to a "Verified" state. Conversely, users can remove existing accounts. This strict management ensures that funds can only be withdrawn to legitimate, confirmed destinations, preventing redirection to unauthorized accounts.

3.2.4 Module 4: Reinvestment & Compliance

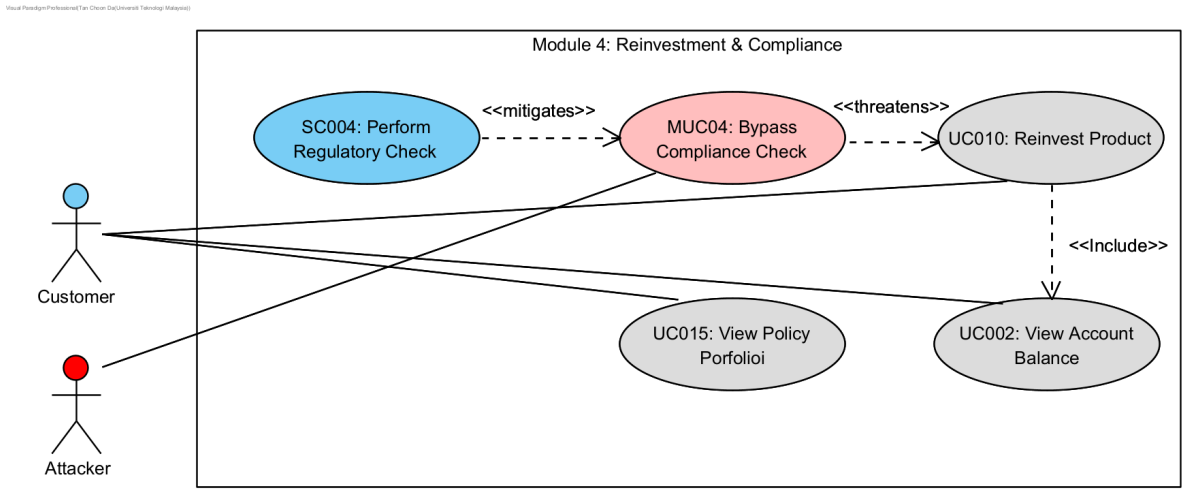


Figure 3.2.4: Use Case Diagram for Module 4

3.2.4.1 UC002 View Account Balance

History Log:	1.0 1.1 1.2 2.0	Create initial use case Fixed pre-conditions Fixed normal flow Added related requirements	
Version:	2.0		
Use Case ID:	UC002		
Use Case Name:	View Account Balance		
Created By:	Lee Ming Da	**Last Updated By:**	Lee Ming Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Customer, Etiqa+ System		
Description:	This use case allows the customer to view their real-time wallet balance by querying the core ledger.		
Pre-conditions:	1. The customer is logged into the Etiqa+ System. 2. The core ledger system is online.		

<table>
<tr><td>Normal Flow:</td><td>1. Customer navigates to "eMoney Dashboard".

2. The system requests current wallet balance.

3. System queries core ledger for funds.

4. System verifies retrieval is successful (Decision: Yes).

5. The system retrieves Balance and Currency.

6. System formats balance for display.

7. The system displays real-time balance.

8. Customer views account balance.</td></tr>
<tr><td>Alternative Flow:</td><td>-</td></tr>
<tr><td>Exception:</td><td>E.1 Retrieval Failed (Decision: No)

1. In Step 4, if retrieval is not successful.

2. System displays "Balance Unavailable".

3. System logs system error.

4. Use Case ends.</td></tr>
<tr><td>Post-conditions:</td><td>Customers have viewed their current financial standing.</td></tr>
<tr><td>Related Requirement:</td><td>
<table>
<tr><th>ID</th><th>Requirement</th><th>Priority</th></tr>
<tr><td>FR-UC002-01</td><td>The system SHALL query the Core Ledger System to retrieve the current available balance associated with the user’s ID.</td><td>High</td></tr>
<tr><td>FR-UC002-02</td><td>The system SHALL display the balance formatted with the correct currency symbol (MYR) and two decimal places.</td><td>High</td></tr>
<tr><td>QR-UC002-01</td><td>The system SHALL retrieve and display the balance within 2 seconds of the dashboard loading (Performance).</td><td>Medium</td></tr>
<tr><td>QR-UC002-02</td><td>If the Core Ledger is unreachable, the system SHALL display a "Balance Unavailable" message rather than crashing or showing zero (Reliability).</td><td>High</td></tr>
</table>
</td></tr>
</table>

	CR-UC002-01	The balance displayed MUST represent the real-time "Available Balance" excluding any frozen or pending funds (Data Integrity).	**Critical**

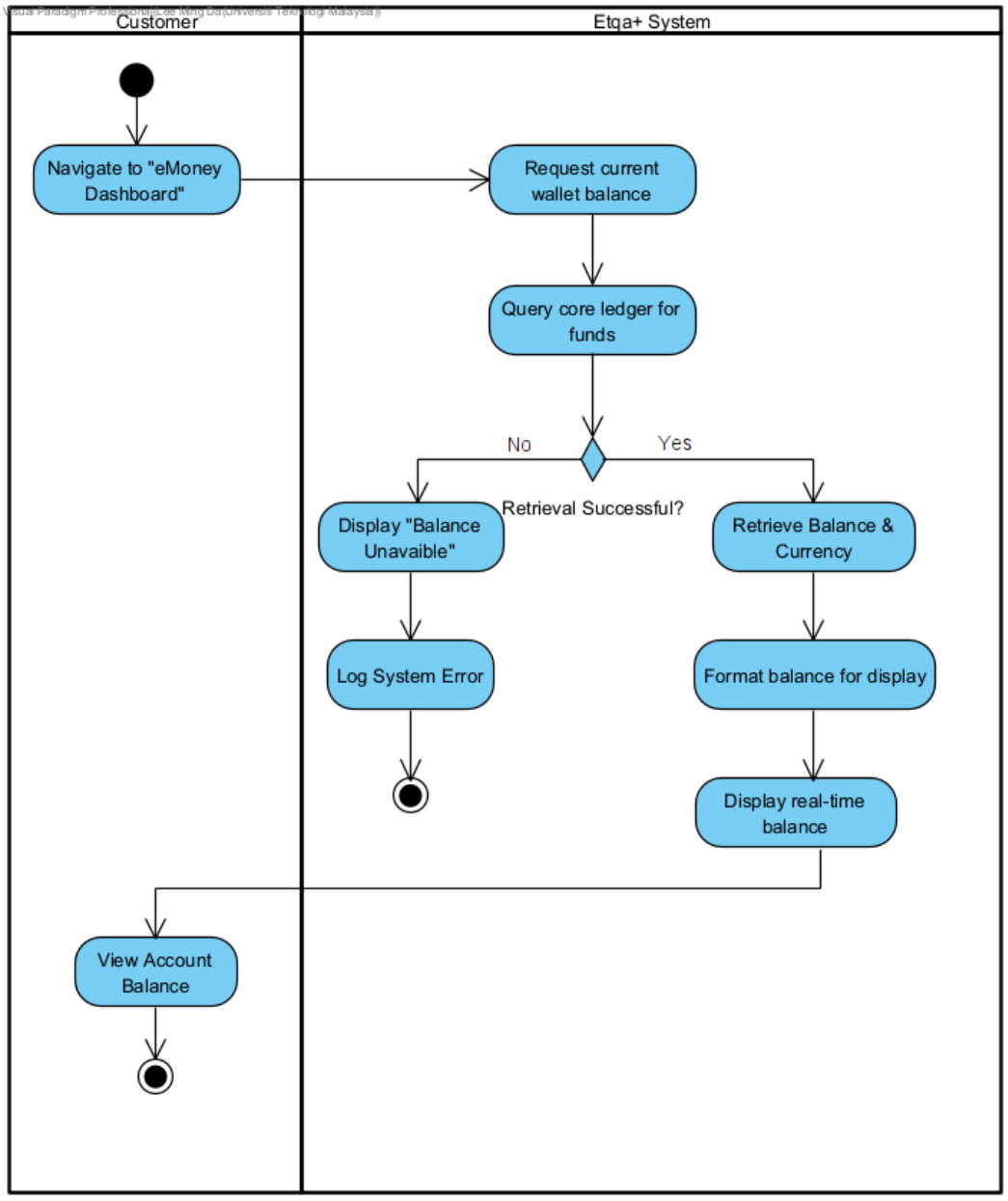


Figure 3.2.4.1: Activity Diagram for UC002 View Account Balance

For UC002, this use case ensures customers are never left guessing about their money. In the old system, funds could be "in transit" with no visibility, but this feature gives users a real-time, accurate view of their wallet balance directly on the dashboard. By querying Etiqa's central Core Ledger, the app shows the exact amount available the moment a claim is paid out or a purchase is made. This immediate feedback loop eliminates anxiety for the customer and significantly reduces the need for them to contact support just to check if a payment has arrived.

3.2.4.2 UC010 Reinvest Product

History Log:	1.0 1.1 1.2 2.0	Create initial use case Fixed pre-conditions Fixed normal flow Added related requirements	
Version:	2.0		
Use Case ID:	UC-010		
Use Case Name:	Reinvest Product		
Created By:	Lee Ming Da	**Last Updated By:**	Lee Ming Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Customer, Etiqa+ System		
Description:	This use case allows the customer to browse products and reinvest funds after passing balance and regulatory checks.		
Pre-conditions:	1. The customer is logged in. 2. The dashboard is accessible.		
Normal Flow:	1. Customer selects "Reinvest" from Dashboard. 2. Customer browses available Etiqa products. 3. Customer enters investment amount. 4. The system retrieves product details. 5. The system calculates Total Premium. 6. The system is called **UC002: View Account Balance**. 7. System validates Balance >= Premium (Decision: Yes). 8. System calls **UC011 (SC004): Perform Regulatory Check**. 9. System validates Regulatory Check Passed (Decision: Yes). 10. The system deducts the amount from the wallet.		

	11. The system creates new policy records. 12. System generates transaction receipt 13. Customer receives success notification.
Alternative Flow:	-
Exception:	**E.1 Insufficient Balance** 1. In Step 7, if Balance < Premium. 2. System displays error: "Insufficient Balance". 3. Use case ends. **E.2 Regulatory Check Failed** 1. In Step 9, if the Regulatory Check fails. 2. System displays error: "Transaction Blocked". 3. Use case ends.
Post-conditions:	Wallet balance is deducted, and a new policy record is created.

Related Requirement:

ID	Requirement	Priority
FR-UC010-01	The system SHALL retrieve and display a list of eligible Etiqa products available for reinvestment.	**Medium**
FR-UC010-02	The system SHALL calculate the total premium based on the product selected and the user's coverage input.	**High**
FR-UC010-03	The system SHALL automatically invoke UC002 (View Balance) to verify the user has sufficient funds before allowing the transaction.	**High**
FR-UC010-04	The system SHALL invoke SC004 (Regulatory Check) to screen the transaction against AML/CFT limits before deduction.	**Critical**
FR-UC010-05	Upon success, the system SHALL immediately deduct the amount from the	**High**

		eWallet and generate a new Policy Record in the database.	
	CR-UC010-01	The system SHALL block the transaction and display "Insufficient Balance" if the wallet funds are lower than the calculated premium.	**High**

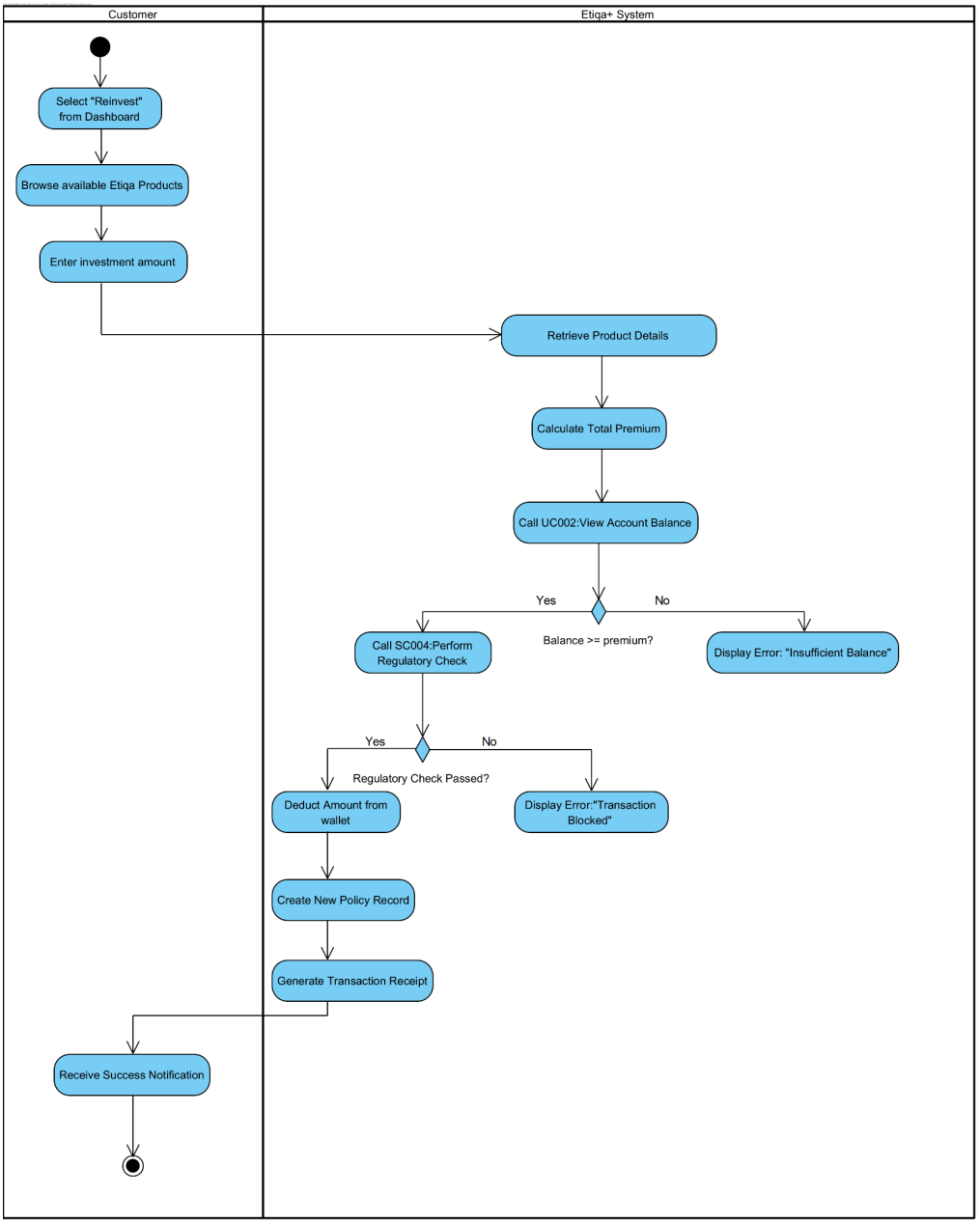


Figure 3.2.4.2: Activity Diagram for UC010 Reinvest Product

For UC010, this use case is designed to create a seamless experience where customers can easily put their payout funds back to work. Instead of waiting for external bank transfers, users can instantly use their eMoney balance to purchase new insurance or Takaful plans directly. The system handles the heavy lifting in the background.It first checks if the customer is eligible for the plan and confirms they have enough funds. Before the transaction is finalized, the system automatically performs a regulatory check to ensure the purchase complies with strict financial guidelines. The result is an instant, secure update to their coverage and wallet balance, making it easier for customers to stay protected.

3.2.4.3 UC015 View Policy Portfolio

History Log:	1.0 1.1 1.2 2.0	Create initial use case Fixed pre-conditions Fixed normal flow Added related requirements	
Version:	2.0		
Use Case ID:	UC015		
Use Case Name:	View Policy Portfolio		
Created By:	Lee Ming Da	**Last Updated By:**	Lee Ming Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Customer, Etiqa+ App		
Description:	This use case allows the customers to view a consolidated list of their active insurance policies and investment products.		
Pre-conditions:	1. The customer is logged in. 2. Customers have at least one active policy/investment.		
Normal Flow:	1. Customer selects "My Portfolio". 2. System requests active policies from Database. 3. The system retrieves policy details. 4. System checks if Policies are Found (Decision: Yes). 5. The system calculates Total Portfolio Value. 6. The system displays a Portfolio Dashboard. 7. The customer selects a specific policy. 8. The system shows detailed breakdown.		
Alternative Flow:	-		

<table>
<tr><td>Exception:</td><td colspan="3">E.1 No Active Investments

1. In Step 4, if no policies are found.

2. The system displays a "No Active Investments" message.

3. Use Case ends.</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">Customers have successfully viewed their portfolio summary or specific policy details.</td></tr>
<tr><td rowspan="6">Related Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>FR-UC015-01</td><td>The system SHALL retrieve all policies with "Active" status linked to the currently logged-in Customer ID.</td><td>High</td></tr>
<tr><td>FR-UC015-02</td><td>The system SHALL calculate and display the "Total Portfolio Value" by summing the coverage amounts/cash values of all active policies.</td><td>Medium</td></tr>
<tr><td>FR-UC015-03</td><td>The system SHALL allow the user to click on a specific policy to view detailed breakdown (Coverage type, Expiry Date, Premium).</td><td>Basic</td></tr>
<tr><td>SR-UC015-01</td><td>The system SHALL mask sensitive policy details such as full beneficiary NRICs on the main dashboard view for data privacy.</td><td>High</td></tr>
<tr><td>QR-UC015-01</td><td>The system SHALL display a "No Active Investments" message if the query returns an empty list, rather than an empty table.</td><td>Basic</td></tr>
</table>

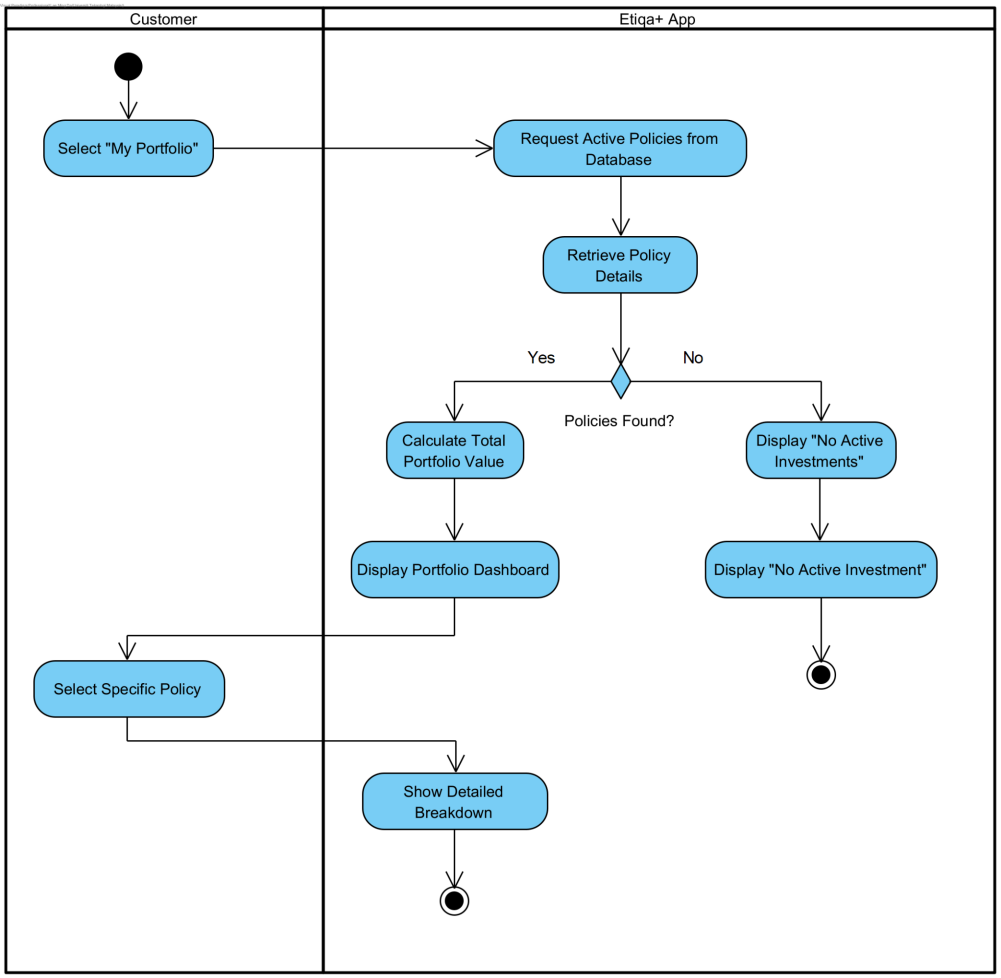


Figure 3.2.4.3: Activity Diagram for UC015 View Policy Portfolio

For UC015, this use case is designed to give customers a comprehensive, real-time overview of their insurance and Takaful assets directly through the Etiqa+ app. Upon accessing the portfolio section, the system automatically queries the central database to retrieve all active policies linked to the user's profile. It then aggregates these details to calculate the total current value of their holdings, presenting the data in a unified dashboard that allows users to drill down into specific policy coverage and premiums. This consolidated view ensures customers can effortlessly monitor their investment performance and protection status without needing to track multiple physical documents.

3.2.4.4 MC004 Bypass Compliance Check

History Log:	0.1 0.2	Create initial misuse case Updated Misuse Flow and Mitigation Flow based on Activity Diagram MUC04	
Version:	1.0		
Misuse Case ID:	MUC04		
Misuse Case Name:	Bypass Compliance Check		
Created By:	Lee Ming Da	**Last Updated By:**	Lee Ming Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Misuser(s)/Threat Agent(s):	Attacker		
Description:	An attacker attempts to bypass the regulatory reporting limit by splitting a large total reinvestment amount into smaller, less suspicious batches.		
Threat:	**Regulatory Evasion / Money Laundering:** Exploiting transaction thresholds to move illicit funds without triggering standard alerts.		
Triggers	Attacker initiates "Submit Reinvest Request #n".		
Pre-Condition(s):	1. Attacker has a valid account with funds. 2. Attacker has identified the system's "Reinvestment Reporting Limit".		
Misuse Flow (Attack Steps):	1. Attacker identifies the "Reinvestment Reporting Limit". 2. Attacker splits the total reinvestment amount into small batches. 3. Attacker submits **Reinvest Request #n**.		

<table>
<tr><td></td><td colspan="3">4. System executes UC010: Reinvest Product.

5. System triggers Call SC004: Perform Regulatory Check.</td></tr>
<tr><td>Mitigation Flow(s):</td><td colspan="3">M.1 System Detects and Blocks Transaction (SC004 Rejected)

1. In the "SC004 Result?" decision step, the system returns Rejected.

2. System executes Block UC010.

3. System executes Freeze User Account.

4. The system sends a "Transaction Blocked" alert to the attacker.

5. The attack fails.</td></tr>
<tr><td>Exception Flow(s):</td><td colspan="3">E.1 False Positive (Legitimate User Blocked)

1. If a legitimate user is flagged by SC004 erroneously.

2. System freezes account (as per Mitigation Flow).</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">The transaction is completed, and illicit funds are successfully reinvested.</td></tr>
<tr><td>Related Use Case(s):</td><td colspan="3">UC010: Reinvest Product</td></tr>
<tr><td rowspan="4">Security Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>SR-MC004-01</td><td>Structuring Detection: The system SHALL calculate the cumulative value of all reinvestment transactions performed by a user within a rolling 24-hour window, not just individual transaction amounts.</td><td>Critical</td></tr>
<tr><td>SR-MC004-02</td><td>Reporting Limit Enforcement: If the cumulative total exceeds the BNM reporting threshold, the system SHALL automatically trigger the Regulatory Check (SC004).</td><td>Critical</td></tr>
<tr><td>SR-MC004-03</td><td>Automated Freeze: If a user attempts multiple transactions that sum up to exactly just below the threshold (Pattern</td><td>High</td></tr>
</table>

		Recognition), the system SHALL flag the account as "Suspicious" and freeze funds immediately.	
	SR-MC004-04	**Audit Trail:** The system SHALL log every blocked attempt with a specific "AML_Risk" tag for review by the Compliance Officer.	**High**

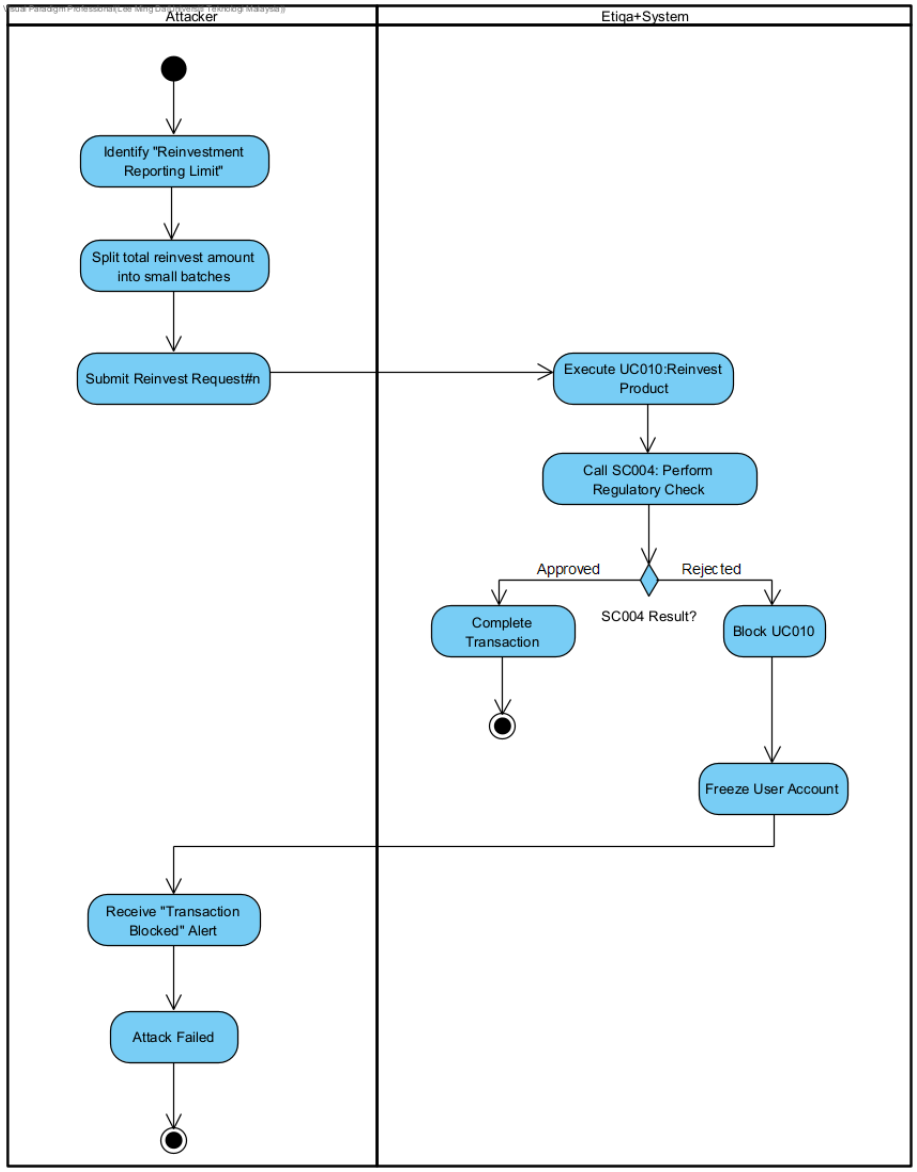


Figure 3.2.4.4: Activity Diagram for MUC04 Bypass Compliance Check

For MUC04, this misuse case describes a structuring attempt where a malicious attacker tries to circumvent regulatory reporting limits. The attacker intentionally splits a large transaction value into multiple smaller, inconspicuous amounts to avoid triggering the automated high-value alerts. This scenario highlights the necessity of the system's advanced monitoring logic, which must detect cumulative transaction patterns over time rather than just isolated events, ensuring that illicit funds cannot flow through the Etiqa ecosystem undetected

3.2.4.5 SC004 Perform Regulatory Check

History Log:	1.0 1.1 1.2 2.0	Create initial use case Fixed pre-conditions Updated Normal Flow to match "Cumulative Total" calculation in activity diagram Added related requirements	
Version:	2.0		
Use Case ID:	SC004		
Use Case Name:	Perform Regulatory Check		
Created By:	Lee Ming Da	**Last Updated By:**	Lee Ming Da
Date Created:	22 December 2025	**Last Revision Date:**	29 December 2025
Actors:	Payout Service System, Compliance System		
Description:	This use case validates reinvestment requests by checking if the user's cumulative transaction total exceeds the daily limit.		
Pre-conditions:	1. A Reinvestment Request (UC010) has been initiated. 2. The Compliance System is online.		
Normal Flow:	1. Payout Service System receives Reinvest Request. 2. System extracts User ID and Transaction Amount. 3. Compliance System retrieves User's Reinvestment History. 4. The system calculates Cumulative Total. 5. System verifies Cumulative Total is within Daily Limit (Decision: No, Total < Limit). 6. System returns status: **Approved**. 7. Payout Service System allows UC10 to proceed.		
Alternative Flow:	-		

<table>
<tr><td>Exception:</td><td colspan="3">E.1 Limit Exceeded (Decision: Yes)

1. In Step 5, if Cumulative Total > Daily Limit.

2. Compliance System returns status: Rejected.

3. Payout Service System blocks UC10.

4. Use Case ends.</td></tr>
<tr><td>Post-conditions:</td><td colspan="3">The transaction is either approved for processing or blocked due to compliance violation.</td></tr>
<tr><td rowspan="6">Related Requirement:</td><td>ID</td><td>Requirement</td><td>Priority</td></tr>
<tr><td>FR-SC004-01</td><td>The system SHALL retrieve the user’s transaction history for the current day to calculate the cumulative total.</td><td>High</td></tr>
<tr><td>FR-SC004-02</td><td>The system SHALL compare the transaction details against the Global AML/CFT Watchlist database in real-time.</td><td>Critical</td></tr>
<tr><td>FR-SC004-03</td><td>The system SHALL return a binary status ("Approved" or "Rejected") to the parent process (UC010) immediately.</td><td>High</td></tr>
<tr><td>SR-SC004-01</td><td>Fail-Safe: If the Compliance System is offline or unreachable, the system SHALL default to "Rejected" (Block Transaction) to prevent regulatory breach.</td><td>Critical</td></tr>
<tr><td>CR-SC004-01</td><td>All regulatory check logs must be retained for a minimum of 7 years as per Bank Negara Malaysia requirements.</td><td>Critical</td></tr>
</table>

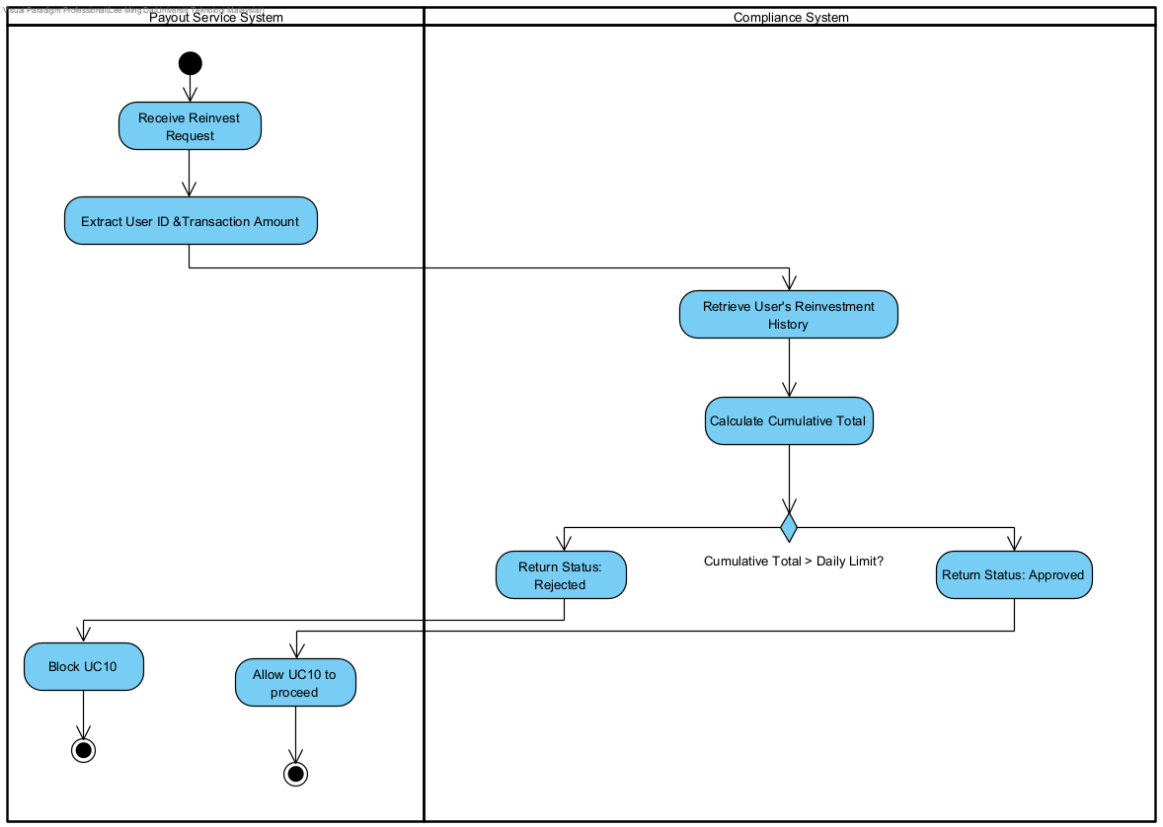


Figure 3.2.4.5: Activity Diagram for SC004 Perform Regulatory Checking

SC004: Perform Regulatory Checking This use case functions as the platform's automated compliance guardian, executing invisibly in the background during critical financial transactions such as reinvestments. Its primary role is to enforce Bank Negara Malaysia's strict regulatory standards by instantly cross-referencing transaction data against global Anti-Money Laundering (AML) and Counter-Financing of Terrorism (CFT) watchlists. Additionally, the system evaluates the activity against the Risk Management in Technology (RMIT) framework to detect any potential fraud or structuring patterns. If a risk threshold is breached or a watchlist match is found, the system immediately halts the transaction, ensuring the platform remains secure without disrupting legitimate user activities.

3.3 Performance Requirements

3.3.1 Response Time (QR UC-002-01):

The system shall retrieve and display the user's Account Balance (UC002) within 2 seconds of the request under normal load conditions.

3.3.2 Throughput:

The system shall support at least 1,000 concurrent users performing active transactions (e.g., withdrawals or document uploads) without service degradation.

3.3.3 Transaction Processing Speed (QR UC-007-02):

The Update User Balance (UC014) process must conclude within 1 second after receiving a success signal from the payment gateway to ensure real-time data consistency.

3.3.4 Availability:

The system shall maintain an uptime of 99.9% during business hours (8:00 AM - 8:00 PM) to ensure accessibility for payouts.

3.3.5 Scalability:

The database shall be capable of handling a 20% month-over-month growth in transaction logs (Transaction History) without impacting query performance.

3.4 Design Constraints

3.4.1 Architectural Standard:

The system must be built using a Microservices Architecture to ensure that the *Payout* Service, Compliance Service, and Document Management Service can be scaled or maintained independently.

3.4.2 Database Design:

The Core Ledger must use a Relational Database Management System (RDBMS) to ensure referential integrity for financial records. NoSQL may be used only for non-critical logs.

3.4.3 Language Requirements:

The backend logic must be implemented in a type-safe language to reduce runtime errors in financial calculations.

3.4.4 Mobile Compatibility:

The frontend of Etiqa+ App must be responsive and compatible with the latest two major versions of iOS and Android operating systems.

3.5 Software System Attributes

3.5.1 Reliability:

The system must implement automated Rollback Mechanisms. If a step in a transaction fails, the system must reverse the previous actions or flag the record for manual reconciliation to prevent financial loss.

3.5.2 Security (Access Control):

All critical financial functions must be protected by Multi-Factor Authentication (MFA).

3.5.3 Maintainability:

The system code must follow industry standard coding guidelines with comprehensive documentation to facilitate future updates by new developers.

3.5.4 Portability:

The server side components should be containerized to allow deployment across different cloud environments without code modification.

3.6 Other Requirements

3.6.1 Audit Logging (FR SC-03-05):

The system shall create an immutable audit trail for every financial transaction and security event, recording the Timestamp, User ID, IP Address, and Action Details.

3.6.2 Data Privacy (PDPA Compliance):

The system must redact or mask sensitive user information on all user interfaces to comply with the Personal Data Protection Act.

3.6.3 Localization:

The system interface shall support both English and Bahasa Melayu to cater to the diverse user base in Malaysia.

4.0 Conclusion

In a nutshell, this Software Requirements Specification (SRS) has successfully defined the comprehensive functional and non-functional requirements for the Etiqa eMoney Payout System. By detailing the system's architecture across four critical modules which is Account & Security, Claims Processing, Withdrawals and Reinvestment, this document serves as a unified blueprint for bridging the gap between Etiqa's business objective and technical implementation.

The proposed system addresses the critical need to modernize insurance disbursements by replacing the legacy 3-day manual transfer cycle with instant, automated digital crediting. This transition not only enhances customer satisfaction through real-time transparency but also significantly improves operational efficiency by reducing manual reconciliation workload. Furthermore, the strict adherence to Bank Negara Malaysia's RMIT guidelines and the integration of Multi-Factor Authentication (MFA) and Fraud Detection mechanisms ensure that the system is resilient against cyber threats and financial abuse.

Moving forward, this SRS document will function as the primary reference for the System Design and Implementation phases. It provides the Development Team with clear logic for backend processes and API integrations, while offering the Quality Assurance (QA) Team precise criteria for validation. Ultimately, the successful deployment of the Etiqa eMoney Payout System will mark a significant milestone in Etiqa's digital transformation journey, delivering a seamless and secure financial experience to policyholders.

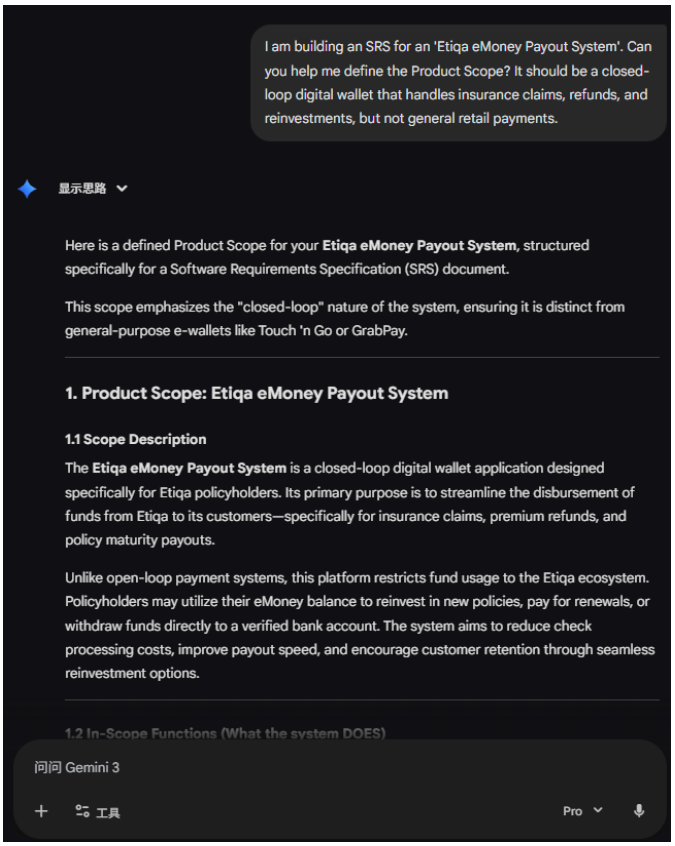
Video Presentation Link

5.0 Appendices

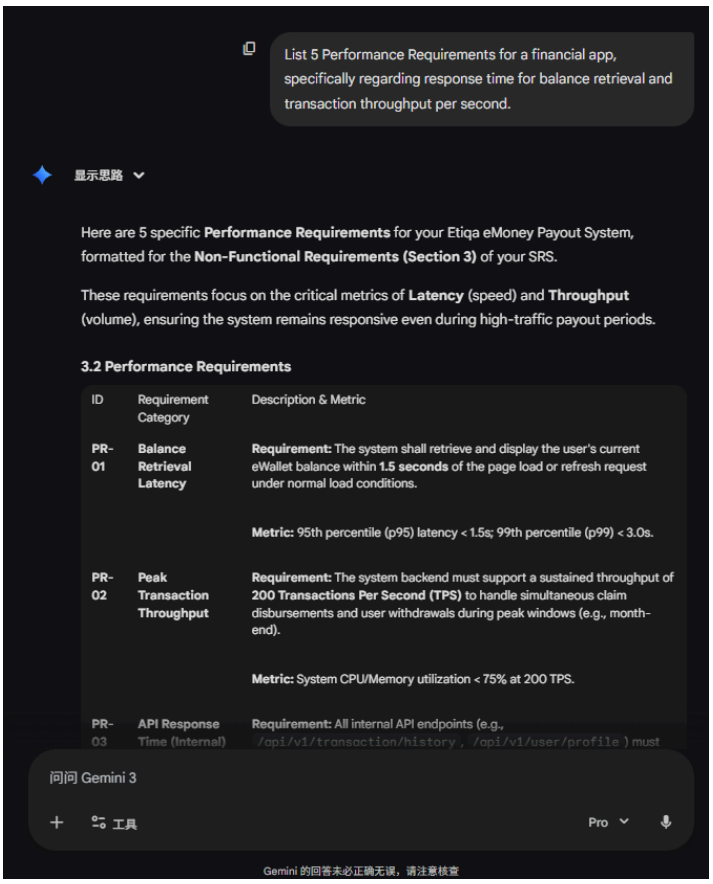
- ***Appendix A: Originality/Similarity Report***



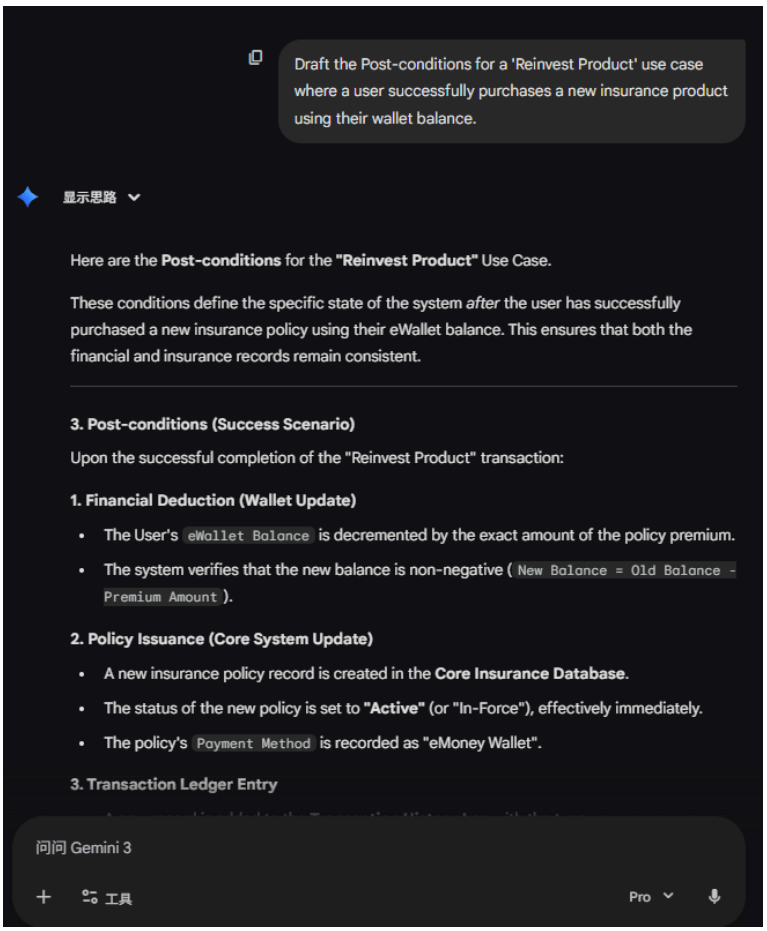
- ***Appendix B: List of Prompts Used***



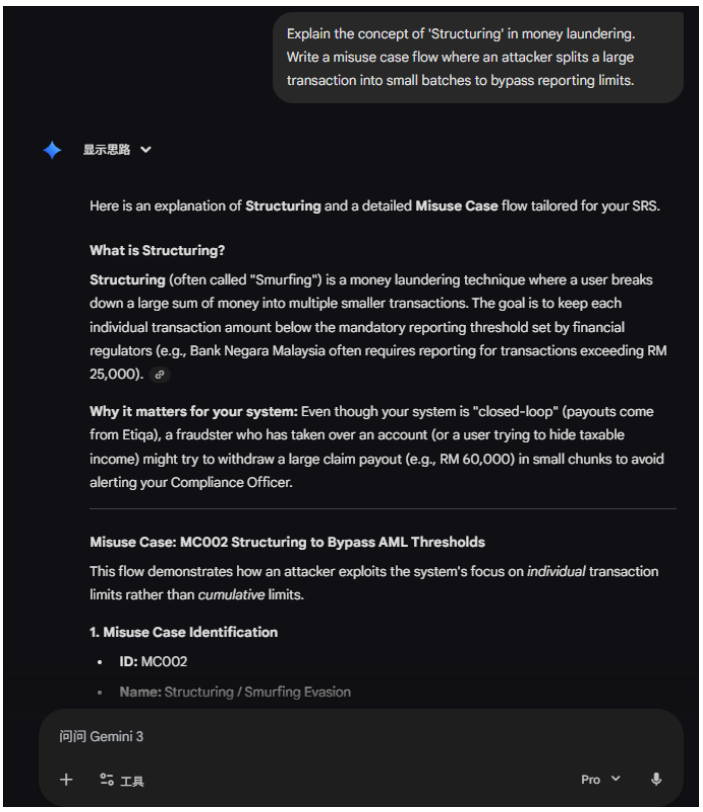
Screenshot of asking Generative AI Chat to define the Product Scope.



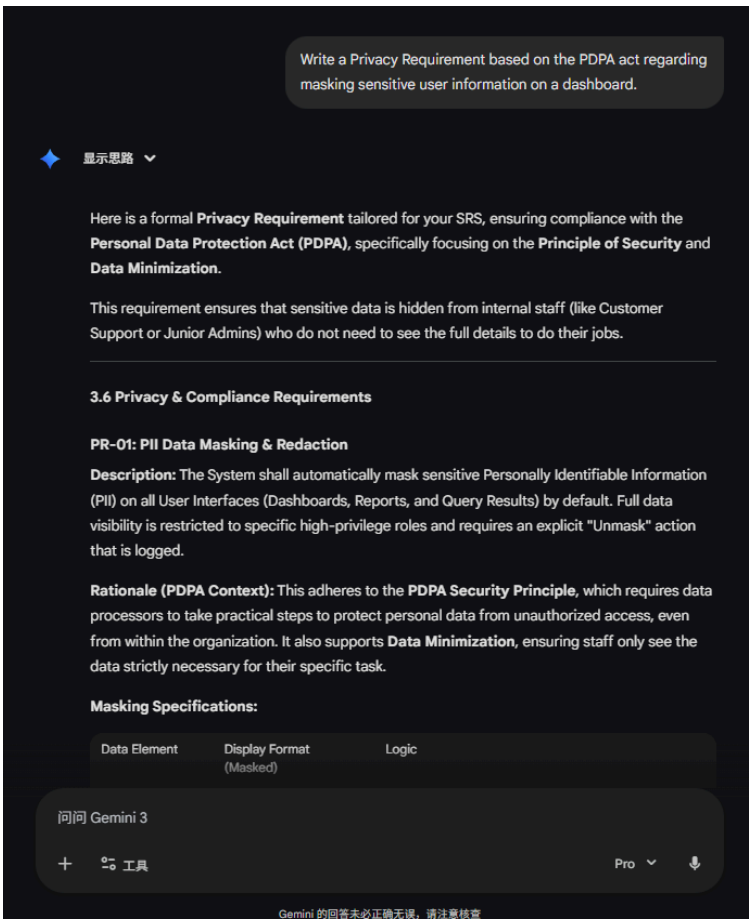
Screenshot of asking Generative AI Chat to suggest Performance Requirements



Screenshot of asking Generative AI Chat to Draft the Post-conditions for a 'Reinvest Product' use case



Screenshot of asking Generative AI Chat to Explain the concept of 'Structuring' in money laundering.



Screenshot of asking Generative AI Chat to suggest Privacy Requirement based on the PDPA act

URL: https://gemini.google.com/share/4c4324746714